Health Insurance Claims Systems Architecture

What a CTO needs to know — using Gravie as the case study. Architecture, data flows, regulations, build vs buy, vendor landscape, and modernization strategies for self-funded health plans.

Audience: VP/CTO-level engineers

Duration: 2-3 hours

Modules: 8

Case Study: Gravie (TPA / Self-Funded Plans)

MODULE 1 **Claims 101 — The Lifecycle of a Health Claim**

Every health insurance claim represents a financial and clinical transaction that touches a half-dozen systems, three or more legal entities, and a regulatory framework spanning federal ERISA statute, state insurance law, and CMS standards. Before you can architect a claims system, you need to understand what a claim actually *is* — not abstractly, but mechanistically. This module walks through the full lifecycle, the key players, the wire formats, and the structural distinctions between carrier models that every CTO in health-tech needs to internalize.

The Full Claim Lifecycle: From Doctor Visit to Payment

A health insurance claim begins the moment a member presents their insurance ID card at a provider's front desk. That ID card encodes a group number (the employer's plan), a member ID, and a payer ID — typically a five-character code that tells the provider's practice management system which clearinghouse or payer to route the electronic claim to. What follows is a deterministic pipeline that, in a well-run system, completes in under 14 calendar days.

Step 1 — Service Delivery & Documentation: The provider renders care and documents it using standardized codes. A physician visit gets coded with an ICD-10 diagnosis code (what's wrong) and CPT procedure codes (what was done). The front desk collects a

copay if applicable, which becomes relevant during adjudication because it counts toward accumulators.

Step 2 — Claim Creation: The provider's practice management system (Epic, Athenahealth, eClinicalWorks, etc.) generates a claim. For a physician, this is a CMS-1500 form (or its electronic equivalent, the 837P transaction). For a hospital, it's a UB-04 (837I). The claim contains member demographics, the rendering provider's NPI, date of service, diagnosis codes, procedure codes, billed charges, and place of service code.

Step 3 — Clearinghouse Submission: The provider almost never submits directly to the payer. The claim goes to a clearinghouse — a transaction hub like Change Healthcare (now Optum), Availity, or Office Ally — which validates the claim format (is the NPI valid? are the code combinations plausible?), translates it to the correct EDI format, and routes it to the correct payer. This is where the 837 transaction lives.

Step 4 — Intake & Acknowledgment: The payer's claims intake system receives the 837, validates it structurally, assigns a claim number (ICN — Internal Control Number), and returns a 277CA (Claim Acknowledgment) back through the clearinghouse. If the claim fails front-end edits (missing required fields, invalid NPI), it's rejected here — not denied. Rejections are formatting/submission failures; denials are clinical/coverage decisions. This distinction matters enormously operationally.

Step 5 — Eligibility & Enrollment Check: The adjudication engine first asks: was this member enrolled in an active plan on the date of service? This requires querying the eligibility system with the member ID and date of service. If eligibility fails, the claim is denied with a specific reason code (CO-4, CO-27, etc.). At Gravie and similar modern TPAs, real-time eligibility verification via 270/271 transactions happens before the claim even reaches adjudication.

Step 6 — Benefits Determination: Given the plan the member was enrolled in on the date of service, what benefits apply? Is this service covered? Is the provider in-network? Is a referral or prior authorization required? The benefits configuration engine evaluates the claim against the plan's benefit design — deductibles, copays, coinsurance, out-of-pocket maximums, and exclusions.

Step 7 — Pricing & Repricing: For in-network providers, payment is determined by the contracted rate. The repricing engine takes the provider's billed charge and replaces it with the negotiated rate. For out-of-network providers, different logic applies — usual and

customary (U&C) rates, reference-based pricing, or balance billing rules depending on plan design. This is where companies like MultiPlan operate, doing out-of-network repricing.

Step 8 — Accumulator Application: Member cost-sharing is calculated by applying the repriced amount against current accumulator balances. If a member has a $2,000 deductible and has already met $1,500, the claim may be partially applied to the deductible and partially covered at the coinsurance rate. Accumulators are running ledgers — deductible met, out-of-pocket met, copay counts — and must be updated atomically with claim adjudication.

Step 9 — Adjudication Decision: The claim resolves to one of several states: paid (approved with payment amount), denied (with denial reason code), pended (needs manual review, additional information, or COB resolution), or adjusted. The system writes the adjudicated claim record with all financial details.

Step 10 — Payment & Remittance: Payment is made to the provider (or member for reimbursement claims) via check or EFT. The 835 Electronic Remittance Advice is generated, detailing exactly how each claim line was processed — what was paid, what member owes, and why any amounts were adjusted. The Explanation of Benefits (EOB) is the consumer-facing version of the 835.

> 💡 **CTO Talking Point:** The average fully-insured claim takes 14-30 days end-to-end. Modern self-funded TPAs like Collective Health and Gravie target under 10 days. The bottleneck is almost never the adjudication engine — it's data quality at intake. Dirty data from provider submissions, enrollment lag from employers, and authorization mismatches account for 60-70% of claims that require human touch. If you're building a claims system, instrument every rejection and pend reason from day one. That telemetry is your product roadmap.

Key Entities in the Claims Ecosystem

Understanding who the players are — and where their financial interests diverge — is essential for system design. Every integration point in a claims system corresponds to an inter-entity data exchange.

Member: The individual covered by the health plan. Members can be employees (subscribers) or their dependents. The member's plan enrollment record is the anchor for all claims processing — every adjudication decision traces back to what benefits that specific member had on that specific date of service. Members interact with the system indirectly through EOBs and through portals (Gravie's member app, Oscar's digital front door).

Provider: The entity rendering care — physicians, hospitals, labs, imaging centers, etc. Providers submit claims and receive payment. They are identified by their NPI (National Provider Identifier), a 10-digit number assigned by CMS. In-network providers have contracts with the payer that define negotiated rates; out-of-network providers do not. The distinction between rendering provider, billing provider, and referring provider matters for claims routing and payment.

Employer / Plan Sponsor: For self-funded plans (which cover over 60% of commercially insured Americans), the employer is the actual insurer — they take on the financial risk. The employer defines benefit design, contributes to a funding account, and pays claims. They contract with a TPA to administer the plan. This is the core of the employer health benefit market where Gravie operates.

TPA (Third Party Administrator): The operational engine of self-funded plans. The TPA processes claims, manages provider networks, handles member services, and provides reporting to the employer. The TPA does not bear insurance risk — they're a services organization. Gravie, Collective Health, and Imagine360 are modern TPAs. Legacy TPAs include Meritain Health, HealthSCOPE, and EBMS.

PBM (Pharmacy Benefit Manager): A specialized TPA for pharmacy claims. PBMs negotiate drug pricing with manufacturers, manage formularies, and process pharmacy claims at point of sale. The big three — CVS Caremark, Express Scripts (Cigna), and OptumRx — control over 80% of the market. PBM integration is a critical and notoriously opaque integration point in any full claims system.

Stop-Loss Carrier: For self-funded employers, stop-loss insurance limits catastrophic exposure. Specific stop-loss covers individual members who exceed a threshold (e.g., $150,000/year). Aggregate stop-loss covers the entire plan if total claims exceed a percentage of expected claims. Stop-loss carriers like Sun Life, HM Insurance, and Tokio

Marine receive claims data feeds and adjudicate their own reimbursement to the employer. This is a separate data flow that claims systems must support.

Clearinghouse: The translation and routing layer between providers and payers. After the Change Healthcare cyberattack in 2024, the single-point-of-failure risk of clearinghouse concentration became acutely visible. Every claims engineering team should understand which clearinghouses their system connects to and have contingency routing.

Claim Types: Professional, Institutional, Pharmacy, Dental, Vision

Professional Claims (CMS-1500 / 837P): Used by physicians, outpatient clinics, labs, ambulances, and other non-facility providers. The key fields are the service lines (up to 6 on a paper form, more electronically), each with a CPT/HCPCS procedure code, ICD-10 diagnosis code pointer, units, and billed charge. The rendering and billing NPI are both present. Professional claims are the highest volume claim type in most systems.

Institutional Claims (UB-04 / 837I): Used by hospitals, skilled nursing facilities, home health agencies, and other institutional providers. The UB-04 form is far more complex — it includes revenue codes (which categorize services by type regardless of CPT), condition codes, occurrence codes, value codes, and span billing for multi-day stays. The accommodation code identifies the type of bed. Institutional claims typically have higher dollar values and more complex adjudication logic.

Pharmacy Claims: Processed in real-time at point of sale using NCPDP (National Council for Prescription Drug Programs) transactions — specifically the D.0 standard. The claim identifies the drug by its NDC (National Drug Code), the dispensing pharmacy by its NCPDP provider ID, and captures quantity, days supply, DAW (Dispense As Written) code, and the prescriber's NPI. Pharmacy adjudication involves formulary lookup, prior authorization check, and benefit copay application — all in under 2 seconds at the pharmacy counter.

Dental Claims (ADA Claim Form / 837D): Use CDT (Current Dental Terminology) procedure codes instead of CPT. Dental plans have distinct benefit structures — usually 100/80/50 coverage tiers for preventive/basic/major services, with annual maximums rather than out-of-pocket maximums. Many health TPAs carve out dental to specialized administrators like MetLife or Delta Dental, but integrated platforms need to handle it.

Vision Claims: The simplest claim type by volume and complexity, but distinct enough to require separate processing logic. Vision plans often use a vendor network (VSP, EyeMed) and have frame/lens allowances rather than traditional benefit structures. The claim format is typically professional but with specialized codes.

EDI Transactions: The Wire Protocol of Health Insurance

All electronic data exchange in U.S. health insurance runs over ANSI X12 EDI standards, mandated by HIPAA. Understanding the transaction set numbers is table stakes for any engineer in this space. The six transactions you will encounter daily:

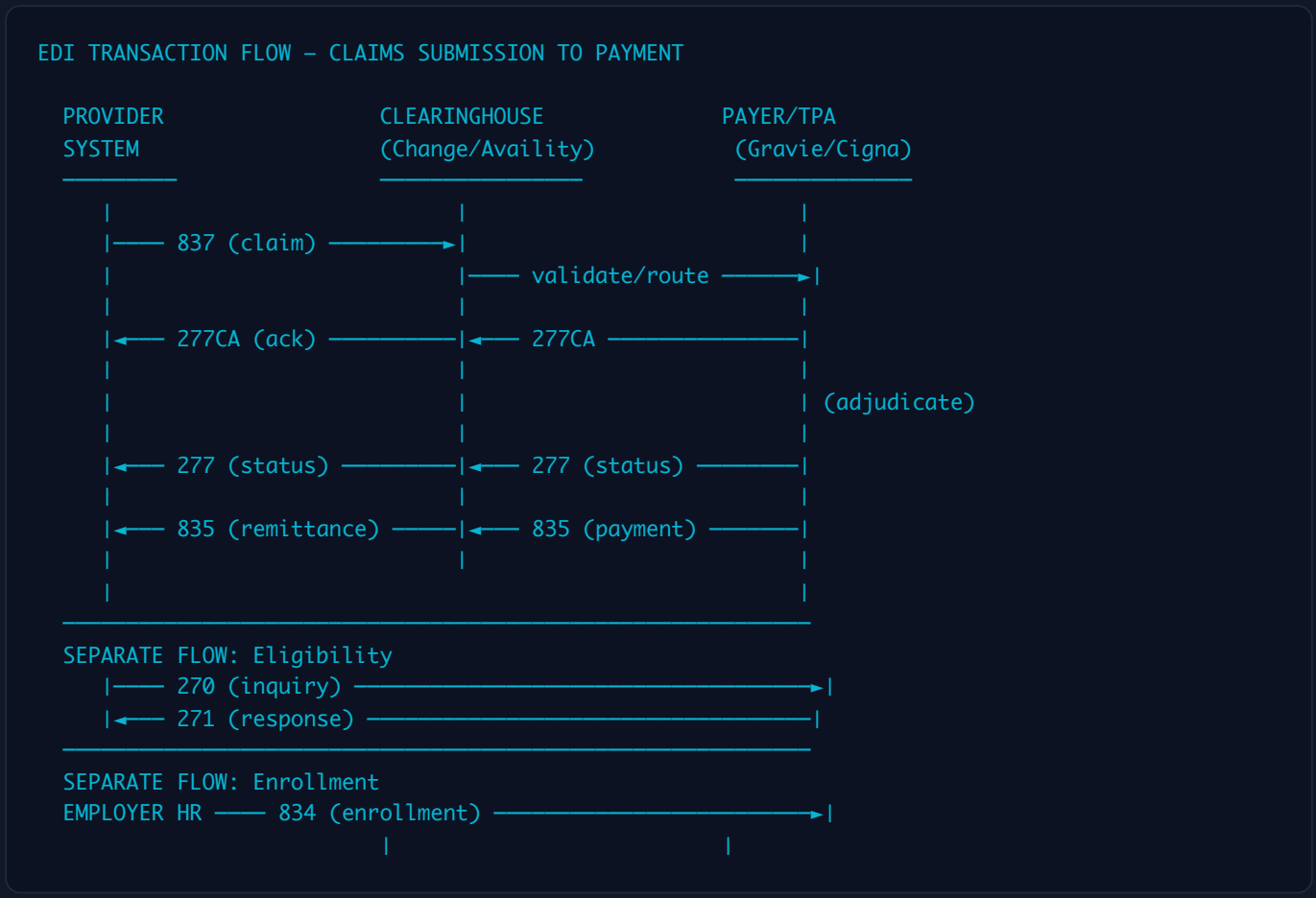
837 (Health Care Claim): The claim submission transaction. 837P for professional, 837I for institutional, 837D for dental. Each 837 is a batch file containing one or more claims, encoded in a hierarchical loop structure using ISA/GS envelope segments, BPR/NM1/CLM segments for claim data, and SV1/SV2 segments for service lines. The format is notoriously verbose and requires specialized EDI parsers.

835 (Health Care Claim Payment/Advice): The remittance advice accompanying payment. Contains CAS (Claim Adjustment Reason) codes and RARC (Remittance Advice Remark Codes) explaining every dollar adjustment. Building an 835 parser that correctly reconciles payment against submitted claims is one of the hardest integration problems in health-tech.

834 (Benefit Enrollment and Maintenance): How employers transmit member enrollment to payers/TPAs. An 834 contains adds, changes, and terminations for members and their dependents. Enrollment lag — when a member is active at the employer but the TPA hasn't received the 834 yet — is the single largest source of eligibility denials and is a constant operational headache.

270/271 (Health Care Eligibility Benefit Inquiry and Response): Real-time eligibility verification. The provider's system sends a 270 asking "is this person covered and what are their benefits?", and the payer responds with a 271 containing coverage details, copay amounts, deductible status, and network indicators. Target response time for 271s is under 3 seconds. Oscar Health and Clover Health have invested heavily in real-time 271 accuracy as a differentiator.

276/277 (Health Care Claim Status Request and Response): Providers check claim status — is it received, pended, paid, or denied? The 276 is the inquiry; 277 is the response. Modern payer portals have largely replaced EDI 276/277 for high-volume billers, but EDI status checks remain critical for automated billing systems.



Real-Time vs. Batch Processing

Health insurance claims processing exists on a spectrum from true real-time (pharmacy point-of-sale, eligibility verification) to overnight batch (most medical claims adjudication) to weekly batch (stop-loss reporting, analytics). The architecture implications are significant.

Real-time processing is required for pharmacy claims (sub-2-second response at the counter), eligibility checks (provider needs an answer before the patient leaves), and authorization decisions (pre-service approvals). Real-time systems require high-availability infrastructure, deterministic latency guarantees, and synchronous integrations.

Batch processing handles the bulk of medical claims adjudication. Claims are ingested throughout the day, accumulated into batches, and adjudicated in processing windows — typically nightly or multiple times per day. This model allows for more complex adjudication logic, cross-claim analysis (coordination of benefits), and cost-effective infrastructure scaling. Most legacy TPAs run nightly batch windows; modern platforms like Collective Health have moved to near-real-time adjudication for most claim types.

The key engineering tradeoff: real-time requires pre-computed state (benefits, accumulators, provider network status must be available in milliseconds), while batch allows reprocessing and correction. A hybrid architecture — real-time intake with async adjudication and real-time status updates — is the current best practice for new builds.

> 💡 **CTO Talking Point:** The shift from batch to near-real-time adjudication is where modern platforms like Collective Health differentiated from legacy TPAs. But "real-time" is a marketing term — what matters is latency SLAs by claim type. Pharmacy must be real-time. Medical can be 24-48 hours and members won't notice. The bigger win from modernization is not speed but *observability* — knowing where every claim is in the pipeline at any moment, and surfacing that to members and providers via APIs.

Carrier vs. TPA vs. ASO vs. MGU — Critical Structural Differences

These four acronyms describe fundamentally different business and risk structures, yet they're often conflated. The distinction determines regulatory requirements, data ownership, and financial flows — all of which have direct system architecture implications.

Model	Risk Bearer	Regulator	Premium Flow	Claims Data Ownership	Examples
Fully Insured Carrier	Insurance company bears all risk	State insurance dept + CMS	Employer pays premium to carrier; carrier pays claims from own funds	Carrier owns data; limited employer access	UnitedHealthca Anthem, Cigna (fully insured lines)
TPA (Third Party Administrator)	Employer bears risk (self-funded)	DOL / ERISA (federal); limited	Employer funds claims account; TPA	Employer owns data; TPA is custodian	Gravie, Collect Health, Meritai HealthSCOPE

Model	Risk Bearer	Regulator	Premium Flow	Claims Data Ownership	Examples
		state oversight	draws against it per claim		
ASO (Administrative Services Only)	Employer bears risk; carrier provides admin services only	DOL / ERISA + carrier's state license	Same as TPA; employer self-funds	Shared — employer owns, carrier administers with contractual data use rights	Cigna ASO, Aet ASO, BCBS AS arrangements
MGU (Managing General Underwriter)	Carrier/reinsurer bears risk; MGU underwrites and administers on their behalf	State insurance dept (bound to carrier's license)	Premium to MGU; MGU remits to carrier after commission	Carrier owns; MGU has operational access	Various specialty/stop-loss MGUs; sor level-funded products

The practical implication for system architects: if you're building for a self-funded TPA model (Gravie's core business), the employer is your true customer, and your data model must accommodate employer-level configuration, employer-level reporting, and employer ownership of claims data. If you're building for a fully-insured carrier, the data model is simpler but regulatory compliance requirements are substantially higher. ASO arrangements add complexity because the carrier's systems may be the system of record for some functions while your platform handles others.

Level-funded plans — a hybrid gaining significant market share among small employers — act like self-funded plans administratively but include an embedded stop-loss layer that looks more like fully-insured to the employer. Level-funded is where Gravie and competitors like Sana Benefits compete heavily, and it requires hybrid system capabilities: TPA-style adjudication with carrier-style reporting and a stop-loss reconciliation layer.

MODULE 2 **The Anatomy of a Claims System** ▶

A claims processing system is not one system — it's a constellation of highly coupled subsystems, each with distinct data models, performance requirements, and failure modes. Legacy platforms (think: FACETS, TriZetto, QicLink) were built as monolithic mainframe applications in the 1980s and 1990s, optimized for batch throughput on expensive hardware. Modern platforms — Collective Health's purpose-built stack, Oscar

Health's in-house system, Clover Health's data-first approach — have rebuilt these capabilities as microservices or modular platforms, but the functional decomposition is strikingly similar across generations. This module maps the full system architecture, the data model, the adjudication engine, and the medical coding universe that every engineer in this space must understand.

Core System Modules

1. Eligibility and Enrollment: The foundational module — every other system depends on it. Stores member records, plan enrollment history, dependent relationships, and coverage effective/termination dates. Receives 834 files from employers (or real-time feeds from HR systems like Workday, ADP, or Rippling via SFTP or API). Must support retroactive enrollment and termination, which cascade into claim re-adjudication. The hardest operational problem in this module is enrollment lag — the 3-7 day window where a member is active at the employer but not yet reflected in the TPA's system. Modern platforms are solving this with direct HRIS integrations rather than waiting for 834 files.

2. Benefits Configuration: Defines what the plan covers and at what cost-sharing level. A plan's benefit configuration is a complex directed graph: service categories → coverage tiers → cost-sharing rules → accumulator linkages → exclusions and limitations. A single employer may have multiple plan options (HDHP, PPO, HMO) each with dozens of benefit configurations. This module is the source of truth that the adjudication engine queries per-claim. Configuration errors here — a miscoded copay, a missing service exclusion — propagate to thousands of claims before discovery. Version control and audit trails for benefit configurations are non-negotiable.

3. Claims Intake: Receives EDI 837 transactions (or increasingly, API-based claim submissions), validates format and required fields, assigns an ICN, and routes the claim to the appropriate adjudication workflow. Intake must handle high throughput (large employers submit hundreds of claims per hour during peak periods), deduplication (providers often resubmit unacknowledged claims), and format normalization (legacy EDI, API JSON, paper-scanned claims). The 277CA acknowledgment is generated here.

4. Adjudication Engine: The core computational module. Takes an ingested claim, applies the full sequence of adjudication rules, and produces an adjudicated claim record with disposition (pay/deny/pend), allowed amount, member liability, and reason codes.

The adjudication engine is typically rule-based — a configurable rule set evaluated in deterministic order. Modern systems like those built by Collective Health layer machine learning on top for fraud detection and auto-adjudication confidence scoring, but the core adjudication logic remains rules-based for auditability and regulatory compliance.

5. Pricing and Repricing: Translates provider billed charges to contracted rates. For in-network providers, this requires a contract database keyed by provider NPI, procedure code, and effective date. For hospitals, contracts are often DRG-based (Diagnosis Related Group) or per-diem rather than per-procedure. For out-of-network, repricing integrates with third-party databases (MultiPlan, FAIR Health, Zelis) or applies reference-based pricing logic. Getting pricing right is where enormous amounts of money are at stake — even 1% error rate on repricing translates to millions of dollars annually for a plan covering 10,000 members.

6. Accumulator Management: Maintains running balances for member cost-sharing components: individual deductible, family deductible, individual out-of-pocket maximum, family out-of-pocket maximum, and any benefit-specific accumulators (e.g., physical therapy visit limits, mental health visit counts). Accumulators must be updated atomically with claim adjudication and must handle retroactive adjustments gracefully. The cross-plan accumulator problem — members switching plans mid-year, or accumulator aggregation across embedded and non-embedded family deductibles — is genuinely complex. Gravie's accumulators module is a key differentiator in their platform; getting it wrong means either over-charging or under-charging members at the worst possible moments (when they're actually sick).

7. Provider Network Management: Stores provider contracts, network participation status, geographic coverage, and credentialing status. This module answers: "Is this provider in-network for this member's plan on this date of service?" The answer determines whether in-network or out-of-network benefit levels apply. Network management is operationally complex — providers join and leave networks, contracts expire and renew, and tiered networks (preferred vs. standard in-network) add further logic. Leased networks (where a TPA accesses a carrier's network like BCBS or Cigna for a fee) require integration with the network owner's directory.

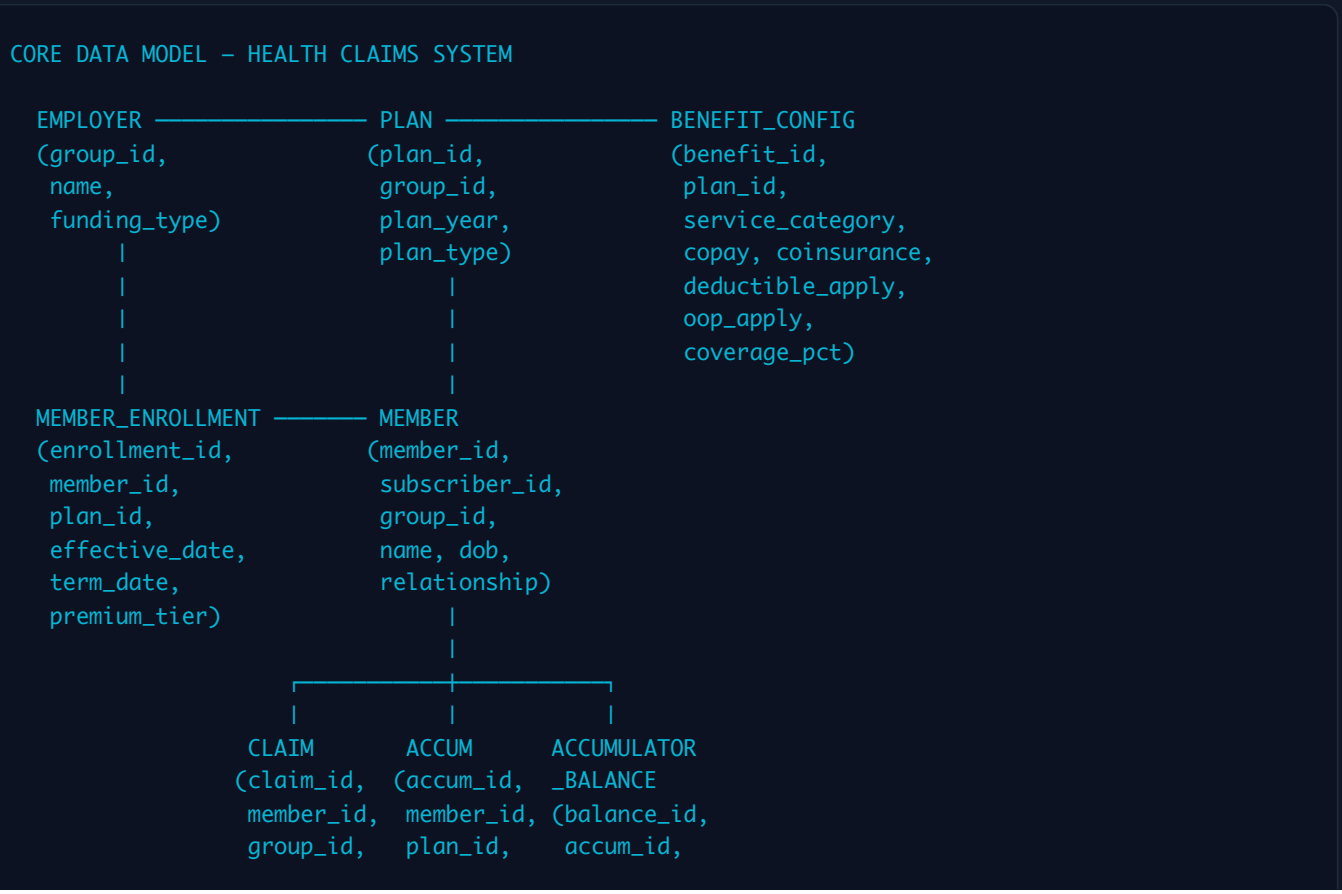
8. Payment and Disbursement: Generates payments to providers (EFT, check, or virtual card) and produces 835 remittance advice. For self-funded plans, this module draws against the employer's claims funding account — requiring integration with the

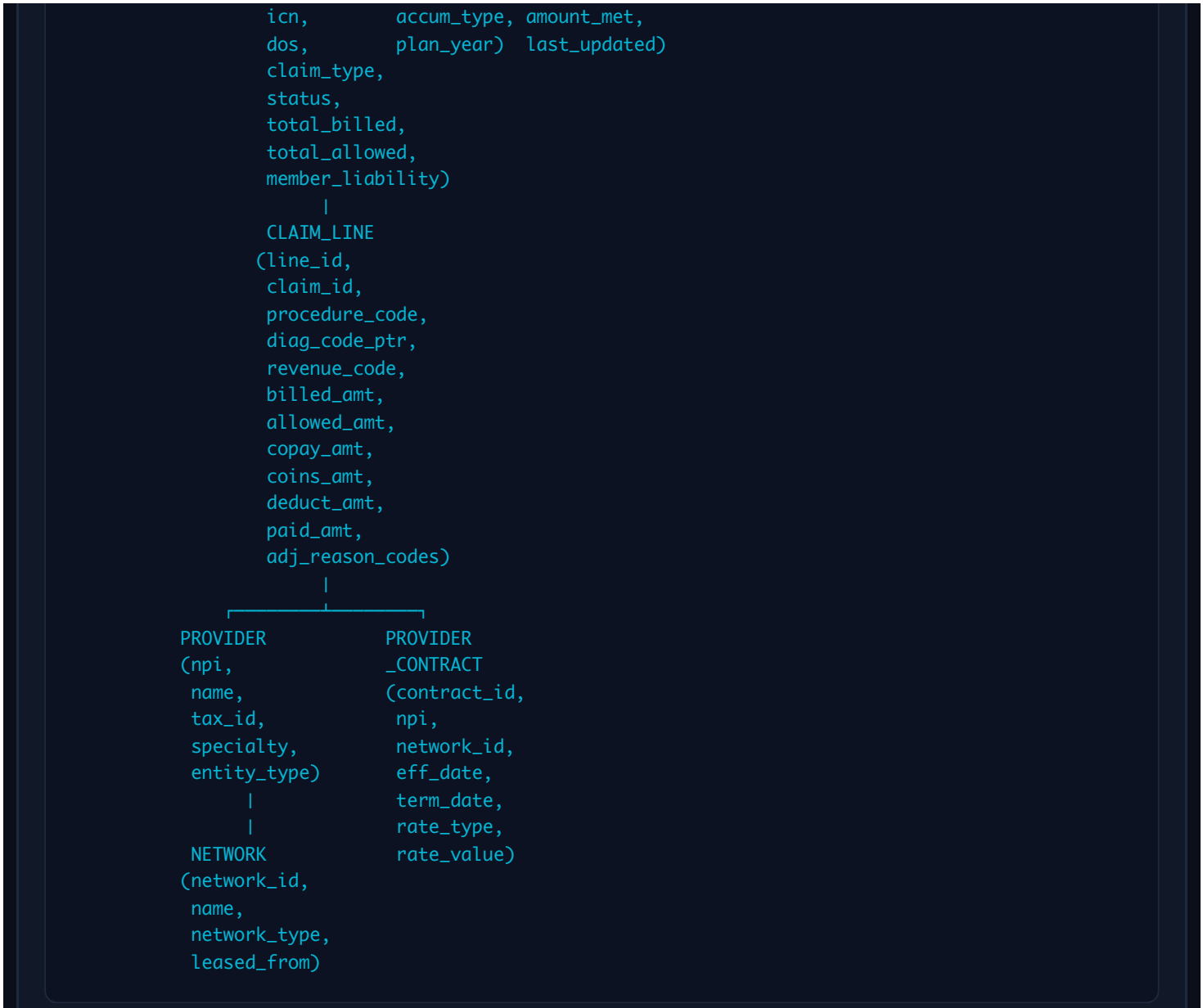
employer's bank account or a custodial account maintained by the TPA. Payment timing is contractually governed (Clean Claims Act requirements in most states mandate payment within 30-45 days for fully insured plans; ERISA governs self-funded). The member-facing component handles reimbursement for out-of-pocket claims and EOB generation.

9. Reporting and Analytics: Produces utilization reports, cost trend analysis, stop-loss bordereau (the detailed claims feed required by stop-loss carriers), employer invoicing, and regulatory reporting. Modern platforms like Clover Health have built significant ML infrastructure on top of claims data for predictive analytics and care gap identification. For self-funded employers, the claims data reporting is a primary reason they chose self-funding over fully-insured — they can actually see where their money is going and take action.

Data Model: Core Entities and Relationships

The data model of a claims system is deceptively complex because of the temporal dimension — almost every entity has effective dates, and queries must be temporally scoped ("what was this member's benefit configuration on this date of service?"). Below is the core entity-relationship structure:



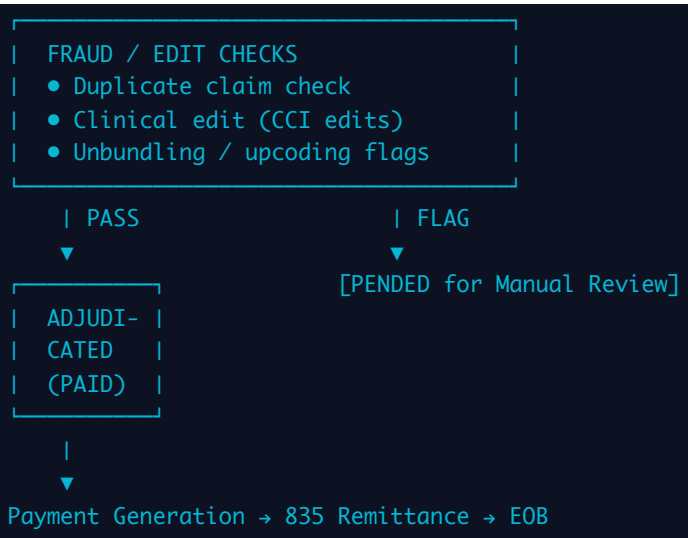


A few critical design decisions embedded in this model: First, the `CLAIM_LINE` is the atomic unit of adjudication — each service line on a claim is adjudicated independently, though cross-line logic (global surgical packages, bilateral procedure modifiers) requires claim-level context. Second, accumulators are keyed to both member and plan year — when a member switches from one plan to another mid-year, accumulator balances may or may not transfer depending on plan design. Third, the temporal scoping problem: a claim for a date of service six months ago must be adjudicated against the benefits, network, and accumulator state as of that date — not today's state.

The Adjudication Engine: Flow and Logic

The adjudication engine is the intellectual center of a claims system. It is fundamentally a decision tree executed against a rich context object (the claim enriched with member,

plan, provider, and accumulator data). The rules execute in a defined sequence because order matters — a coverage determination failure preempts the need for pricing logic.



Auto-Adjudication Rates: The Core Operational KPI

Auto-adjudication rate is the percentage of claims that pass through the entire adjudication pipeline without human intervention. It is the single most important operational efficiency metric in a claims system, and it directly determines the cost structure of a TPA. Industry benchmark: 85-95% for mature systems. Legacy TPAs often run at 70-80%. Collective Health has publicly claimed rates in the high 90s for their platform.

Every claim that falls out of auto-adjudication to a human reviewer costs $15-50 in operational expense (industry estimates). On a book of 100,000 members with 1.2 million claims per year, moving from 80% to 95% auto-adjudication saves 180,000 manual reviews — roughly $4-9M annually in just operational labor. The architectural implication: every pend reason is a product defect. Instrument pend reasons obsessively, route them to engineering sprint planning, and eliminate them one by one.

The most common pend triggers: missing prior authorization, COB pending investigation, provider credentialing gap, claim edit (Correct Coding Initiative edits — more on these below), suspected duplicate, and missing clinical documentation. Each of these is a solvable engineering and data problem. Real-time authorization lookup eliminates the auth pend. Real-time credentialing feeds eliminate the credentialing pend. Duplicate detection ML eliminates manual duplicate review.

> 💡 **CTO Talking Point:** Auto-adjudication rate is a lagging indicator — it tells you what happened, not why. The leading indicators are: front-end rejection rate (claims that never

make it to adjudication due to format errors), pend reason distribution (which rules are triggering human review), and mean time to adjudicate (MTTA) by claim type. Oscar Health's engineering team built a real-time ops dashboard tracking all three by employer group. When a new employer's claims started pend-ing at 30% instead of 5%, they knew within hours — not at the monthly operations review. That visibility is the real differentiator.

Medical Coding Systems: ICD-10, CPT, HCPCS, NDC, Revenue Codes, POS

Medical coding is the lingua franca of claims. Every claim line contains codes from one or more of these systems, and correct code interpretation is prerequisite to correct adjudication. Engineers need to understand not just what the codes are, but how they interact and how coding errors create claims processing failures.

Code Set	Maintained By	Format	Used On	Purpose in Adjudication	Update Frequenc
ICD-10-CM / ICD-10-PCS	CMS + CDC	3-7 alphanumeric characters	All medical claims	Diagnosis; drives coverage determination, medical necessity, COB sequencing	Annual (Oct 1)
CPT (Current Procedural Terminology)	AMA	5-digit numeric	Professional (837P)	Procedure identification; primary driver of pricing logic for professional claims	Annual (Jan 1)
HCPCS Level II	CMS	Letter + 4 digits (A0000–V9999)	Professional + some institutional	DME, ambulance, drugs administered in office, supplies; extends CPT coverage	Quarterly + annua
NDC (National Drug Code)	FDA	10 or 11 digits (labeler-product-package)	Pharmacy (NCPDP), some medical	Drug identification for formulary lookup, DAW, generic substitution rules	Continuous (new drugs)
Revenue Codes	NUBC (National Uniform Billing Committee)	3-4 digit numeric	Institutional (837I / UB-04)	Categorizes hospital services by type (pharmacy, room, OR, imaging) for pricing	Annual

Code Set	Maintained By	Format	Used On	Purpose in Adjudication	Update Frequenc
Place of Service (POS)	CMS	2-digit numeric (11=office, 21=inpatient, 22=ER, etc.)	Professional (837P)	Determines site-of-service differential pricing and some coverage rules	Occasional updates

The interaction between these code sets is where complexity explodes. A single claim line may have: an ICD-10 diagnosis pointer (required to link diagnosis to procedure), a CPT code (the procedure), a HCPCS modifier (e.g., -59 for distinct procedural service), a POS code (where it was performed), and a rendering NPI. The Correct Coding Initiative (CCI) maintained by CMS defines bundling edits — pairs of CPT codes that should not be billed together on the same claim because one is included in the other. CCI edits are published quarterly and contain over 200,000 code pair edits. Implementing CCI edits is a non-trivial engineering task but is essential for preventing overbilling and for accurate adjudication.

ICD-10 code updates on October 1st each year add 200-500 new codes and modify hundreds of others. Every claims system must support versioned code tables so that a claim submitted today for a service six months ago adjudicates against the ICD-10 version that was valid six months ago. This is the same temporal scoping problem that appears throughout claims system design.

> 💡 **CTO Talking Point:** Code set maintenance is a continuous operational burden that many engineering teams underestimate. ICD-10 updates October 1st, CPT updates January 1st, HCPCS updates quarterly, and CCI edits drop quarterly. You need a code set management process: automated ingestion from authoritative sources (CMS downloads, AMA licenses), version control, regression testing against historical claims, and deployment pipelines that don't require a code release to update code tables. If updating a code set requires a sprint and a release window, you're going to miss updates and adjudicate claims incorrectly. Gravie and Collective Health both treat code set management as infrastructure, not a feature — it runs on an automated pipeline with alerting when updates are available.

Coordination of Benefits (COB): When Members Have Multiple Plans

Coordination of Benefits (COB) is the process of determining how two or more insurance plans share payment responsibility when a member is covered by more than one plan. COB applies in several common scenarios: a married couple where both spouses carry employer-sponsored coverage and add each other as dependents; a child covered by both parents' plans; a Medicare-eligible retiree who also has employer coverage; or a member with Medicare as secondary.

COB is governed by NAIC (National Association of Insurance Commissioners) guidelines and typically follows the "birthday rule" for dependent children: the plan of the parent whose birthday falls earlier in the calendar year is primary. For spouses covering each other, the member's own plan is always primary. For Medicare, complex CMS sequencing rules apply based on employer size and disability status.

The claims system must: (1) detect that COB applies — typically from enrollment data indicating other coverage or from a claim submission that indicates another payer; (2) determine primary vs. secondary sequencing; (3) adjudicate as primary payer if primary, applying full cost sharing; (4) adjudicate as secondary payer if secondary, applying the "non-duplication" or "maintenance of benefits" method to calculate the secondary payer's liability given what the primary already paid.

COB is one of the top pend reasons in any claims system because it often requires outbound investigation — calling the member or the other payer to verify primary coverage. Modern platforms are working to eliminate this friction through real-time COB databases (CAQH CORE COB Smart transactions, BenefitPoint, Experian Health), which allow automated COB sequencing without manual outreach. Oscar Health and Clover Health have invested in COB automation as part of their broader claims STP (straight-through processing) initiatives.

The financial stakes of COB errors are significant in both directions: incorrectly paying as primary when you're actually secondary means you've overpaid and must pursue recovery from the other payer (a costly and slow process). Paying as secondary when you're actually primary means the member was under-served — their claims went to the wrong insurer first, creating delays and member frustration. Implement COB detection at intake, not at adjudication — the earlier you identify COB complexity, the less downstream disruption it causes.

From a data model perspective, COB requires tracking "other insurance" records on the member, storing primary EOB data when adjudicating as secondary, and maintaining COB decision history for audit. The cross-system complexity of COB — coordination with an external payer that may use entirely different systems and data standards — is a reminder that a claims system is never truly standalone. It is always a node in a larger ecosystem of payers, providers, clearinghouses, and regulatory systems, each with their own data formats and processing timelines.

MODULE 3 **Gravie's Model — Self-Funded Plans & TPAs** ▶

How Self-Funded Plans Work: Employer as Risk-Bearer

In a fully-insured plan, an employer pays a fixed monthly premium to a carrier — UnitedHealth, Aetna, BCBS — and the carrier absorbs all claim risk. The employer's liability is capped and predictable. In a self-funded (self-insured) plan, the employer retains claim risk entirely: when an employee needs a $180,000 NICU stay, the employer's bank account pays it. The monthly premium stream disappears; instead, the employer funds a claims account and draws from it as adjudicated claims are paid. The structural shift is enormous — the employer moves from insurance customer to quasi-insurer.

Administrative complexity follows immediately. A carrier handling a fully-insured book has decades of infrastructure: provider contracting, claim adjudication engines, UR/UM systems, member portals, COB logic, appeals workflows, and 835/EOB generation. An employer going self-funded needs all of that operational capability without the carrier relationship. This is where the Third Party Administrator (TPA) enters. The TPA is the operational surrogate: it holds the plan document, contracts with provider networks (or accesses a leased network like Cigna ORCA or Aetna Signature), adjudicates claims against the benefit structure, issues EOBs, manages appeals, and remits payment from the employer's trust account. Critically, the TPA does not bear claim risk — it is purely a services and technology vendor. Gravie operates as a TPA for its self-funded employer clients.

The financial mechanics are straightforward but the operational mechanics are not. The employer establishes a dedicated claims trust or bank account, typically funded via a per-

member-per-month (PMPM) deposit based on actuarial projections. The TPA draws against this account for approved claims. Cash flow management matters enormously — large claims can hit in clusters, and the employer must maintain sufficient liquidity. Employers under roughly 100 lives typically cannot self-fund viably because variance is too high; the sweet spot for pure self-funding starts around 200-500 lives, and stop-loss insurance extends it downward.

Gravie Comfort: $0 Copay Plan Design From a Claims Perspective

Gravie Comfort is a self-funded plan design built around zero cost-sharing for common, high-value care: primary care visits, preventive services, generic drugs, and mental health visits carry $0 copay and $0 coinsurance. This is not charity — it is value-based plan design grounded in health economics research showing that cost-sharing at the point of care suppresses appropriate utilization and increases downstream costs by delaying treatment. From a claims systems perspective, $0 copay structures actually simplify some adjudication logic while complicating others.

Simplified: member liability calculation for in-scope services is zero. No deductible accumulator to update, no coinsurance calculation, no member-side 835 remittance to generate for those claims. The adjudicator checks benefit eligibility, verifies the service falls within the $0 category (procedure code, revenue code, and provider specialty all matter), and issues payment at 100% of the contracted rate. Complicated: the benefit structure must correctly distinguish $0-copay services from cost-sharing services, which requires a precise benefit configuration table — often called a benefit matrix or benefit grid — mapping CPT codes, revenue codes, and modifier combinations to the appropriate cost-sharing tier. An error in this mapping is catastrophic: either members get billed incorrectly (grievance risk) or the plan overpays (financial loss). Gravie's tech challenge is maintaining this benefit matrix reliably and auditing it continuously.

Gravie Comfort also bundles ancillary benefits — dental, vision, mental health — into the base plan rather than as separate riders. From a claims routing perspective, this means the adjudication system must handle claims across multiple benefit categories with different networks, different fee schedules, and different accumulators, all within a single plan document. The member's $0 copay experience is the output of complex benefit logic running correctly underneath.

> 💡 **CTO Talking Point:** The $0 copay UX that differentiates Gravie Comfort in the market is entirely a function of benefit configuration accuracy in the adjudication engine. If your CPT-to-tier mapping has a 1% error rate and you're processing 100,000 claims per year, 1,000 members get incorrect EOBs. The competitive moat Gravie has built is not just the plan design — it's the operational discipline to run zero-cost-sharing plans at scale without billing errors that erode member trust. This is a data quality and configuration management problem as much as it is an insurance problem.

ICHRA: Individual Coverage HRA Administration

The Individual Coverage Health Reimbursement Arrangement (ICHRA) is a post-2020 benefit structure (formalized by final rule in 2019, effective 2020) that allows employers of any size to give employees a defined tax-free monthly dollar allowance to purchase individual market health insurance — ACA marketplace plans, off-exchange plans, or short-term plans depending on eligibility rules. The employer sets the allowance amount (which can vary by age and family status within IRS rules), the employee buys their own plan, pays their own premiums, and submits for reimbursement through the HRA administrator.

Gravie administers ICHRA for employers who want defined-contribution benefits without managing a group plan at all. The administrative model is fundamentally different from traditional claims adjudication. There are no claims to adjudicate — Gravie's role is: (1) verify that the employee has enrolled in a qualifying individual health plan (substantiation), (2) process reimbursement requests for premiums and eligible out-of-pocket expenses up to the employer-defined allowance, (3) manage the allowance balance and ensure reimbursements do not exceed it, and (4) handle compliance reporting under ACA affordability rules.

The technology stack for ICHRA administration is meaningfully different from group plan TPA tech. The core data objects are allowance accounts (like an FSA), substantiation workflows (verifying plan enrollment documentation), and reimbursement queues rather than claim line adjudication pipelines. Integrations shift too: instead of clearinghouse connections for 837/835 EDI, the critical integrations are with the marketplace (to verify enrollment via CMS APIs), with payroll systems (reimbursements flow through payroll in many configurations), and with document management systems for storing

substantiation evidence. Gravie building ICHRA administration alongside traditional TPA services means maintaining two operationally distinct technology stacks with different compliance profiles.

Stop-Loss Insurance: Specific, Aggregate, and Lasering

Stop-loss insurance is the mechanism that makes self-funding accessible to mid-market employers by capping their downside. Without it, a single catastrophic claim — a premature infant, a transplant, a cancer case — could bankrupt a small employer's claims fund. Stop-loss is an insurance policy purchased by the employer (not by employees) that reimburses the employer when claims exceed defined thresholds. Understanding stop-loss is essential because it directly shapes how TPAs like Gravie must report, aggregate, and track claims data for their employer clients.

Specific stop-loss (also called individual stop-loss) protects against any single member's claims exceeding an attachment point — commonly $50,000 to $250,000 per member per year. Once a member's paid claims cross the specific attachment point, the stop-loss carrier reimburses the employer dollar-for-dollar above that threshold. The TPA's job is to track per-member aggregate paid claims throughout the plan year and trigger the stop-loss reimbursement workflow at the appropriate time. This requires per-member claim accumulation logic that is separate from and parallel to the benefit accumulator logic (deductibles, OOP max) that drives member cost-sharing.

Aggregate stop-loss protects against total plan claims exceeding a percentage — typically 120-125% — of expected claims for the entire group. If the actuary projects $2M in total claims and actual claims hit $2.5M (125% of expected), aggregate stop-loss kicks in for the excess. Aggregate is the "catastrophic year" protection; specific is the "catastrophic individual" protection. Both can be in force simultaneously, which is the norm for employers under ~500 lives.

Lasering is the stop-loss carrier's mechanism for excluding or surcharging known high-cost individuals from coverage. During underwriting, the stop-loss carrier reviews the group's medical history. A member with a $400,000 chronic condition may be "lasered" — their specific attachment point is set at $800,000 instead of $100,000, effectively removing stop-loss protection for that member. For the TPA, lasers appear as member-level exceptions in the stop-loss configuration that must be tracked and correctly applied

when generating stop-loss reports. Failing to track lasers means the employer incorrectly believes they have stop-loss protection they do not have — a fiduciary failure and a financial time bomb.

The TPA Tech Stack: Gravie vs. a Full Carrier

The technology requirements for a TPA and a full carrier overlap significantly but diverge in critical areas. A carrier like UnitedHealth Group runs a vertically integrated stack:

actuarial pricing engines, underwriting systems, group enrollment platforms, fully proprietary adjudication (Optum's ClaimLogiq, internally built systems), pharmacy benefit management (OptumRx), care management platforms, utilization management engines (InterQual integrated), member portals, provider portals, and a massive analytics infrastructure for risk adjustment and population health. The carrier owns the network contracts, maintains the fee schedule database for every CPT × provider × geography combination, and bears the financial risk — so every technology component ties back to managing that risk.

Gravie as a TPA does not bear claim risk, does not maintain primary network contracts (it leases), and does not need actuarial pricing infrastructure for individual risk. Its tech stack is narrower but must be deep in the areas that matter for employer clients and members.

Capability	Full Carrier (UHC/Aetna)	Gravie TPA	Notes
Claims Adjudication Engine	Proprietary, massive scale (100M+ claims/yr)	TPA-grade, configurable benefit matrices	Gravie needs deep configurability for Comfort plan; carriers need raw throughput
Network Contracting	Direct contracts with every in-network provider	Leased networks (Cigna ORCA, Aetna Sig, etc.)	Carrier maintains fee schedules; TPA ingests them
Actuarial/Underwriting Systems	Full suite — risk rating, reserve modeling	Not needed (employer bears risk)	Stop-loss underwriting done by stop-loss carrier
Utilization Management	Proprietary UM engine (InterQual, MCG integration)	Typically outsourced to UM vendor	Prior auth workflows must integrate with adjudication
Pharmacy Benefit Management	Owned PBM (OptumRx, CVS/Caremark)	Carved-out PBM integration (RxBenefits, IngenioRx)	PBM adjudicates separately; TPA reconciles pharmacy claims
Member Portal / Mobile App	Often dated, built for volume not UX	Core differentiator — modern UX, transparency	Gravie's competitive advantage is in this layer

Capability	Full Carrier (UHC/Aetna)	Gravie TPA	Notes
Employer Reporting / Analytics	Standardized reports, difficult to customize	Configurable per-employer reporting	Employers want real-time claims spend visibility
Stop-Loss Tracking	Not applicable (carrier bears risk)	Critical — per-member accumulation, laser tracking	Must be accurate to protect employer against miscalculation
ICHRA Administration	Not offered (incompatible with carrier model)	Full allowance management + substantiation	Gravie's ICHRA capability differentiates from legacy TPAs
EDI Clearinghouse Integration	Owned or deeply integrated (Change Healthcare)	Via clearinghouse partner	837/835/270/271/276/277 all required
Risk Adjustment / ACA Reporting	Full ACA risk adjustment for individual/small group markets	ACA 1094/1095 reporting for employer mandate	Different compliance surface area
COB (Coordination of Benefits)	Full COB with proprietary data sources	COB required, often via CAQH CORE integration	Self-funded plans have same COB obligations

Fiduciary Responsibility Under ERISA

ERISA (Employee Retirement Income Security Act of 1974) governs self-funded employer health plans and imposes fiduciary duties on plan sponsors (employers) and, in some cases, on TPAs. This is not a compliance footnote — it has direct implications for how Gravie must build and operate its systems. An ERISA fiduciary must act solely in the interest of plan participants, must administer the plan in accordance with the plan document, must avoid prohibited transactions (self-dealing), and must carry a fidelity bond. Employers are always plan fiduciaries. TPAs can be fiduciaries depending on the discretionary authority granted to them in the administrative services agreement.

When Gravie makes discretionary claims decisions — approving or denying a claim based on its interpretation of the plan document — it may be acting as a functional fiduciary. This has legal consequences: fiduciaries can be personally liable for breaches, and the

DOL has enforcement authority. In practice, most TPAs carefully draft administrative services agreements to limit discretionary authority and retain plan sponsor decision-making on appeals, specifically to avoid fiduciary status. But the operational reality is that the adjudication engine is making quasi-discretionary decisions constantly — applying clinical editing rules, applying network access logic, applying benefit matrix mappings — and errors in those decisions create fiduciary exposure.

From a technology standpoint, ERISA compliance demands: (1) an immutable audit log of all claim decisions with the rule applied and the data state at time of adjudication, (2) a complete appeals workflow with required response timelines (72 hours urgent care, 30 days standard), (3) accurate plan document enforcement — the system must adjudicate to the plan document, not to a default configuration, and (4) Form 5500 reporting infrastructure. The 5500 is an annual DOL filing that discloses financial information about the plan; for plans over 100 participants, a Schedule C must disclose all service provider fees over $5,000, including TPA fees. This requires the TPA to provide structured financial data to the employer each year.

How Gravie Differentiates Through Member Experience and Plan Design

Legacy TPAs — HealthSCOPE, Allied Administrators, Benefit Administrative Systems — compete on price and operational reliability. They do not compete on member experience. The typical legacy TPA member portal looks like it was built in 2009, requires a member ID card to log in, does not show real-time deductible status, and makes it impossible to find in-network providers without calling a phone number. This is not hyperbole; it is the baseline against which Gravie competes.

Gravie's differentiation strategy operates on two layers. The first is plan design innovation: creating benefit structures like Gravie Comfort that align financial incentives with appropriate care utilization, eliminating cost-sharing barriers for high-value services, and bundling ancillary benefits in ways that simplify the employee experience. The second is technology-enabled transparency: a modern member portal and mobile app that shows real-time claims status, cost-sharing accumulators, provider cost estimates before care, and plain-language benefit explanations. Both layers require the underlying claims data to be accessible, clean, and real-time — which means Gravie's data architecture must treat member-facing applications as first-class consumers of claims data, not an afterthought.

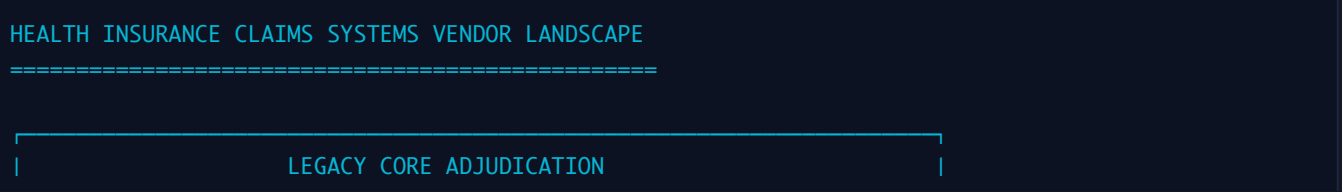
The competitive implication for the tech organization is significant. Member experience investment — mobile app, portal, cost transparency tools — must be treated as product engineering, not IT operations. The claims adjudication engine is infrastructure; the member-facing layer is product. Most legacy TPAs have only infrastructure. Gravie's bet is that the product layer is where employer retention and employee satisfaction are won, and that the infrastructure (adjudication, EDI, stop-loss tracking) must be reliable enough that it never creates surface area for the product to fail.

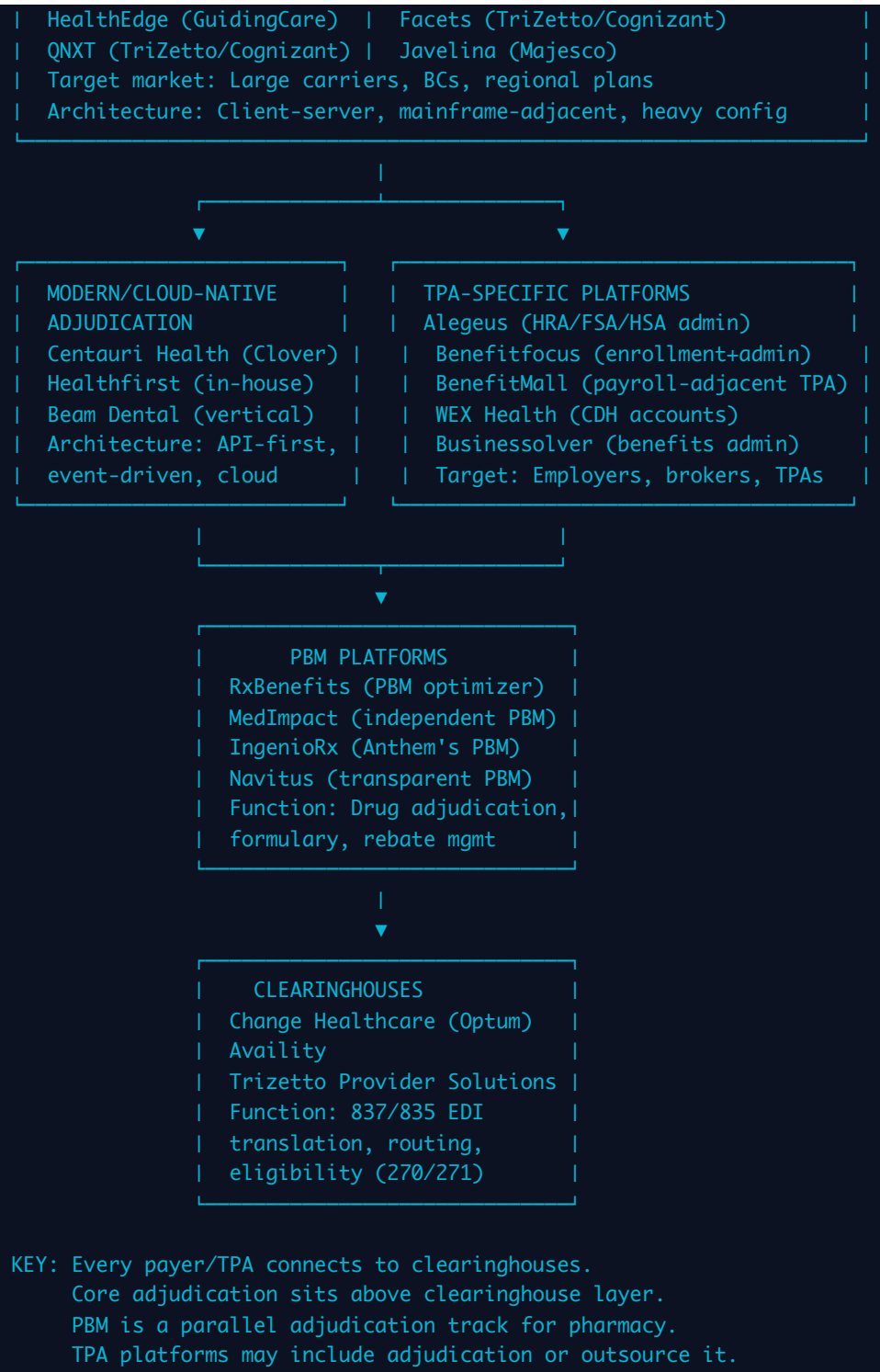
> 💡 **CTO Talking Point:** ERISA's audit and appeals requirements are not just compliance overhead — they are a forcing function for good engineering. A system that cannot produce an immutable, timestamped audit trail of every adjudication decision with the exact rule version applied is not just legally exposed, it is operationally blind. When an employer asks "why was this claim denied?" you need to replay the exact state of the adjudication engine at the moment the decision was made — not today's rule set applied retroactively. Immutable event sourcing or an append-only audit log is the architectural pattern that satisfies both ERISA and good engineering simultaneously.

MODULE 4 **Build vs Buy — The CTO's Decision Framework** ▶

The Vendor Landscape: A Taxonomy of Claims System Suppliers

The health insurance claims systems vendor market is stratified across at least five distinct categories, each targeting a different buyer profile, technical architecture era, and functional scope. Before any build-vs-buy decision, a CTO must map the landscape accurately — conflating a clearinghouse with a core adjudication system, or a PBM platform with a TPA administration platform, leads to catastrophic misscoping. The landscape has also shifted meaningfully post-2018 as cloud-native entrants have challenged incumbents who built on mainframe and client-server architectures in the 1990s and have been accruing technical debt ever since.





Legacy core adjudication vendors — Facets (TriZetto, now Cognizant), QNXT (also TriZetto), HealthEdge, and Javelina — dominate the installed base at large carriers and Blue plans. Facets alone is estimated to run behind 25-30% of commercial health plan claims in the US. These systems are extraordinarily capable in terms of feature breadth: they handle medical, dental, vision, pharmacy (via PBM integration), complex COB scenarios, capitation, subcapitation, and virtually every HIPAA transaction set. They are also 20-30 years old in their core data models, require armies of configuration specialists to implement, and are notoriously difficult to integrate with modern APIs. Implementations

routinely run 18-36 months and $10-50M for large carriers. For a company like Gravie — which needs speed, configurability, and a modern API surface — these systems are architectural liabilities masquerading as features.

Modern cloud-native adjudication options are still sparse but growing. Clover Health built Centauri internally and has open-sourced components. Healthfirst (a not-for-profit insurer) built significant proprietary claims infrastructure. Beam Dental built a vertically integrated dental claims and payments platform from scratch. The pattern across these is: companies with unusual plan designs, specific member populations, or strong technology cultures who could not find a vendor that met their needs built their own. This is Gravie's peer cohort for build-decision reference points.

TPA-specific platforms — Alegeus, WEX Health, Benefitfocus, BenefitMall — are built for the employer benefits administration market rather than carrier-grade adjudication. Alegeus is the dominant HSA/FSA/HRA administration platform and would be the obvious buy for Gravie's ICHRA administration layer. WEX Health similarly focuses on consumer-directed health accounts. Benefitfocus and Businessolver are benefits administration platforms that handle enrollment and eligibility but not claims adjudication. The mistake CTOs make is conflating benefits administration (enrollment, eligibility, account management) with claims adjudication — they are architecturally and functionally distinct problems requiring distinct systems.

Build Scenarios: When Custom Is Strategically Justified

Building proprietary claims infrastructure is expensive, risky, and slow. It should only be justified when one or more of the following conditions hold: (1) the plan design is so differentiated that no vendor supports it without prohibitive customization, (2) the member experience is the primary competitive moat and the system layer directly determines experience quality, (3) the data architecture needed for analytics and product feedback loops cannot be achieved with vendor-controlled data models, or (4) the organization has a long enough time horizon and sufficient engineering talent to sustain a 5-10 year build investment.

Gravie Comfort's $0 copay structure with bundled ancillary benefits is a genuine build justification case. Legacy adjudication systems like Facets have benefit configuration models built around traditional deductible/coinsurance/copay structures. Configuring a

plan where a specific subset of CPT codes has $0 cost-sharing, where the benefit structure bundling dental and vision into a medical plan is seamless, and where the member-facing experience shows this cleanly requires either building on a flexible platform or fighting against an inflexible one. When "vendor customization" costs more in implementation fees and maintenance complexity than building the capability, build wins on total cost of ownership.

The other strong build signal is when the data produced by the claims system is a core input to product differentiation. If Gravie wants to surface cost transparency tools — "your upcoming knee arthroscopy at TRIA Orthopaedic Center will cost you $0, while the same procedure at Park Nicollet will cost you $240" — that requires real-time access to clean, structured claims and fee schedule data in a data model Gravie controls. Vendor systems typically make this data available via batch extracts (nightly, weekly) in proprietary formats that require substantial transformation. Building the adjudication layer on an architecture designed for event streaming and real-time data access enables the product capabilities that a nightly batch export never will.

Buy Scenarios: When Vendor Solutions Earn Their License Fees

The most expensive mistake health tech CTOs make is not buying when they should — it is building when they should buy, then drowning in maintenance. Claims adjudication has enormous regulatory and operational surface area: HIPAA transaction compliance, COB logic that covers hundreds of edge cases developed over 30 years of case law, Medicare secondary payer rules, ACA mandated benefit requirements (preventive care, mental health parity, no cost-sharing for COVID testing), timely filing limits, prompt payment laws that vary by state, and clinical editing logic (duplicate claim detection, unbundling edits, coding consistency checks). Every one of these is a known, solved problem in mature vendor systems. Building it from scratch means rediscovering every edge case through production failures.

Standard plan administration — PPO plan with standard deductible/OOP/coinsurance, standard network, standard COB — is a clear buy scenario. There is nothing proprietary about adjudicating a $1,500 deductible / 80-20 coinsurance / $5,000 OOP max plan. Buying HealthEdge or a modern equivalent means inheriting decades of edge case handling, regulatory update cycles managed by the vendor, and implementation

playbooks refined across hundreds of clients. The break-even on building vs. buying this type of plan administration capability almost never favors build.

Speed to market is also a decisive buy signal. A healthcare startup entering the TPA market needs to process claims correctly on day one — provider trust is fragile, and a mis-adjudicated claim that results in an incorrect provider payment or member bill creates relationship damage that compounds quickly. A vendor system with a 6-month implementation timeline beats an 18-month build timeline when the market window requires faster entry. Regulatory compliance timelines are non-negotiable — HIPAA, ACA, state prompt payment laws do not flex to accommodate build schedules.

> 💡 **CTO Talking Point:** The cognitive trap in health tech build-vs-buy decisions is mistaking claims adjudication complexity for competitive differentiation. Processing a standard EOB correctly is not a competitive advantage — it is table stakes. Where competitive differentiation actually lives is in plan design flexibility, member experience quality, employer reporting depth, and speed of benefit configuration changes. A pragmatic hybrid approach buys the adjudication commodity and builds the differentiation layer. The question to ask of every build proposal is: "Does building this give us a capability that no vendor provides and that our customers will pay for?" If the answer is not an unambiguous yes, buy.

The Hybrid Approach: Bought Core, Built Differentiation

The hybrid architecture — purchased core adjudication wrapped with custom-built member experience, employer analytics, and benefit configuration tools — is the model that resolves the build-vs-buy tension for most health tech companies at Gravie's stage. The core insight is that adjudication is a function, not a product. The product is everything the member and employer experience around that function: how they find care, understand benefits, see costs, get support, and measure plan value. If the adjudication function is reliable and its data is accessible, everything built on top of it can be proprietary.

The technical requirements for a successful hybrid architecture are: (1) the core adjudication vendor must provide a real-time or near-real-time API for claim status, accumulator balances, and EOB data — not just nightly batch extracts, (2) the vendor's data model must be mappable to an internal canonical data model that the custom-built

layers consume, (3) the vendor must support webhooks or event streaming so that claim events (submitted, pended, approved, denied, appealed) propagate immediately to downstream systems, and (4) the benefit configuration layer must be accessible enough that Gravie's operations team can configure new plan designs without filing a vendor change request. Most legacy systems fail on points 3 and 4. Modern API-first vendors (HealthEdge's newer cloud platform, purpose-built systems like those from Centivo or Imagine360) are better positioned here.

The hybrid model also allows for incremental displacement. Gravie might start with a bought adjudication core for standard plan types, build custom benefit configuration tooling on top, build the member portal and employer dashboard as proprietary product layers, and then selectively replace adjudication components as the business case for each replacement clarifies. This is dramatically lower risk than a big-bang custom build and allows the engineering team to learn the problem domain before committing to proprietary solutions for the most complex components.

Cost Analysis: Build vs. Buy at Scale

Any CTO presenting a build recommendation to a board must come with a rigorous cost model. The numbers below represent industry benchmarks — actual costs vary significantly with engineering market, geographic location, vendor negotiating leverage, and implementation complexity, but the order-of-magnitude ranges are defensible.

A ground-up proprietary claims adjudication system — covering medical claims adjudication, COB, appeals workflow, EDI integration (837/835/270/271), stop-loss tracking, and basic employer reporting — requires a dedicated engineering team of 20-40 engineers over 3 years before the system is production-grade for a mid-market TPA book of business. At fully-loaded engineering costs of $200-350K per engineer per year (senior engineers in health tech in Minneapolis or remote are in this range), that is $12-42M in engineering labor alone. Infrastructure costs, vendor partnerships (clearinghouses, UM vendors, PBM integrations), compliance and legal review, and QA overhead add 30-50% to the labor number. Realistic total: $15-60M over 3 years, with $20-30M being the defensible central estimate for a company of Gravie's scale and ambition.

A purchased core adjudication system from a mid-market vendor (not Facets-tier, but a modern TPA-grade system) carries: implementation costs of $500K-$3M depending on complexity and data migration scope, annual licensing/SaaS fees of $1-5M depending on claims volume (per-claim pricing is common at $0.30-$1.50 per claim), and ongoing customization and integration maintenance of $300K-$1M per year in vendor professional services or internal integration engineering. Realistic total over 3 years: $4-12M, with $6-8M being a defensible central estimate. The buy scenario is 2-4x cheaper over the first 3 years.

The build-vs-buy cost crossover — where cumulative build costs become competitive with cumulative buy costs — typically occurs at year 5-8, assuming the custom-built system achieves full feature parity and does not require significant re-architecture. For companies with 10+ year time horizons, dominant market positions, and unique plan designs, the build investment can be justified. For companies still finding product-market fit or operating in a market where the plan designs are not yet fully differentiated, buying preserves capital for the customer acquisition and plan design innovation that actually drives growth.

Migration Risks: The Hidden Cost of Platform Transitions

Every claims system transition — whether replacing a legacy vendor with a new vendor, replacing a vendor with a custom build, or consolidating multiple systems — carries operational risks that dwarf the license fee and engineering labor costs in potential downside. A claims system migration touches every critical operational relationship: members, providers, employers, stop-loss carriers, and regulators. Getting it wrong means providers don't get paid on time (network damage), members get incorrect EOBs (grievance exposure), stop-loss reporting is disrupted (financial exposure), and regulators notice (compliance exposure). The costs of a failed migration cannot be fully modeled in advance.

Parallel running — operating both old and new systems simultaneously and comparing outputs before cutting over — is the standard risk mitigation approach. It requires running both systems on production claim volumes, reconciling every discrepancy, and building confidence that the new system produces identical outputs to the old system for all claim types before any cutover. Parallel running at meaningful scale typically requires 3-6 months and doubles operational complexity during that window. The engineering cost is

not just the new system — it is the reconciliation tooling, the ops team supporting two systems, and the incident response capacity to handle discrepancies. Budget 20-30% additional engineering capacity during a parallel run period.

Data migration for claims systems is particularly treacherous because historical claims data must remain queryable for COB purposes, appeals, stop-loss reconciliation, and ERISA compliance for up to 7 years after the plan year. Migrating historical claims data to a new system without corrupting accumulator histories, losing member eligibility snapshots, or breaking stop-loss tracking is a data engineering project that commonly takes 6-12 months for a mid-size book of business. The risk of incomplete migration is not just operational — an ERISA audit that reveals incomplete historical records is a fiduciary failure.

Provider disruption is the least-discussed migration risk and often the most consequential. When a TPA changes systems, provider-facing workflows change: the EDI sender/receiver IDs on 835s change, prior authorization submission portals change, provider portal URLs and credentials change, and EOB formats change. Large provider organizations (health systems, large medical groups) have EDI operations teams that must be notified and reconfigured. Small providers (independent practices, rural clinics) often lack EDI sophistication and experience payment delays as a direct result of migration. A single well-regarded health system going to collections against Gravie members during a system migration is a reputational and relationship disaster that dwarfs any technology savings.

Decision Matrix: Scoring the Build vs. Buy Decision

A structured scoring framework prevents build-vs-buy decisions from becoming advocacy exercises dominated by whoever presents most persuasively to leadership. The following matrix covers the decision dimensions that matter for a health insurance claims system context. Score each dimension 1-5 for both build and buy options, weight by strategic importance, and sum the weighted scores. The framework is most useful as a structured conversation tool — the act of scoring forces clarity on assumptions that are often left implicit.

Decision Dimension	Weight	Build Score (1-5)	Buy Score (1-5)	Scoring Guidance
Plan Design Uniqueness	20%	5 — if $0 copay, bundled ancillary, ICHRA hybrid	2 — vendors support standard designs only	Score build higher when plan design has no vendor analog; score buy higher for standard PPO/HMO structures
Time to Market	15%	1 — 18-36 months to production quality	4 — 6-12 months typical implementation	Score build lower whenever market timing is competitive; build timelines are compressible but not below ~12 months for core adjudication
3-Year Total Cost	15%	2 — $15-30M fully loaded	4 — $4-10M implementation + licensing	Adjust for company's capital position; pre-Series B companies should weight this dimension higher
Data Architecture Control	15%	5 — full control of schema, event stream, APIs	2 — vendor-controlled data model, batch exports	Score build higher if real-time member-facing products require direct data access; score buy higher if analytics can tolerate T+1 latency
Regulatory Compliance Coverage	10%	2 — must build all compliance logic from scratch	5 — vendors maintain HIPAA, ACA, state law updates	Compliance surface area in claims is enormous; never underestimate the ongoing cost of maintaining regulatory currency in a custom-built system
Engineering Talent Availability	10%	2 — claims domain engineers are rare and expensive	4 — implementation requires fewer domain specialists	Health insurance claims domain expertise is genuinely scarce; factor in recruiting runway and attrition risk over a multi-year build
Vendor Lock-in Risk	8%	5 — no lock-in, full portability	2 — migration costs are	Facets/QNXT migrations are notoriously difficult; factor in whether the

Decision Dimension	Weight	Build Score (1-5)	Buy Score (1-5)	Scoring Guidance
			prohibitive; vendor has leverage	vendor is a consolidation target (Change Healthcare/Optum example)
Member Experience Control	7%	4 — built for your UX requirements	2 — vendor portals are dated, minimal customization	Hybrid architecture can score 4 for both if vendor provides good API access and member UX layer is custom-built
Scalability (Claims Volume)	5%	3 — depends heavily on architecture choices made	4 — vendors have proven scale across large client books	Modern cloud-native vendors score higher here; legacy vendors have scale but not elastic scale
Operational Risk (Migration/Cutover)	5%	3 — greenfield build avoids migration; subsequent migrations still hard	3 — any platform change carries migration risk	Roughly equal for greenfield; score buy lower if migrating an existing book of business to a new vendor

Interpreting the matrix for Gravie's specific context: the high weights on Plan Design Uniqueness and Data Architecture Control favor build for the benefit configuration and member-facing layers. The high weights on Time to Market, 3-Year Total Cost, and Regulatory Compliance Coverage favor buy for the core adjudication engine. This naturally points toward the hybrid architecture — buy the adjudication commodity, build the differentiation — as the highest-scoring approach when weights are applied to a mid-market, growth-stage TPA with unique plan designs and a modern engineering culture.

Architecture Layer	Recommended Approach	Rationale	Vendor Examples
Medical Claims Adjudication (standard plans)	Buy	Commodity function; massive regulatory and edge-case surface area; no differentiation in correct adjudication of standard claims	HealthEdge (cloud platform), Centivo, purpose-built TPA systems
Benefit Configuration Engine (Comfort-style plans)	Build or buy with deep API access	$0 copay + bundled ancillary = non-standard benefit matrix that most	Custom-built on top of vendor adjudication APIs

Architecture Layer	Recommended Approach	Rationale	Vendor Examples
		vendors cannot represent cleanly; this is where plan design IP lives	
ICHRA Administration	Buy (account management), Build (workflow layer)	Alegeus/WEX handle HSA/HRA account mechanics; but ICHRA substantiation workflows and employer-facing tooling need customization	Alegeus (accounts), custom workflow layer
PBM Integration	Buy (carve-out PBM)	Drug formulary management, rebate contracting, pharmacy network — no startup-scale company should build this	RxBenefits, MedImpact, Navitus
EDI Clearinghouse	Buy (always)	837/835 transaction processing is pure infrastructure; clearinghouses have network effects with thousands of providers already connected	Availity, Change Healthcare (Optum)
Member Portal + Mobile App	Build	Primary competitive differentiation surface; requires real-time claims data access; vendor portals are architecturally incompatible with modern UX standards	Custom React/mobile + vendor APIs
Employer Reporting + Analytics	Build	Employers want configurable, real-time claims spend visibility; vendor reporting is static and inflexible; analytics is a retention driver	Custom data warehouse + BI layer (dbt + Snowflake + Looker)
Stop-Loss Tracking	Build (thin layer on adjudication data)	Stop-loss configuration is per-employer with laser exceptions; vendor systems support it poorly;	Custom accumulation service consuming adjudication events

Architecture Layer	Recommended Approach	Rationale	Vendor Examples
		the financial stakes of errors are too high to accept vendor black-box behavior	
Prior Authorization / UM	Buy	Clinical criteria (InterQual, MCG) are regulated and require continuous clinical maintenance; outsource to a UM vendor	Cohere Health, Evicore, Carelon
Appeals Workflow	Build (thin) or configure vendor workflow	ERISA-mandated timelines and audit requirements need reliable workflow; can be built as a thin service on top of adjudication events if vendor workflow is insufficient	Custom or vendor workflow module

💡 **CTO Talking Point:** The most important build-vs-buy decision at Gravie is not "adjudication engine or not" — it is "what is the data architecture that enables the products we want to build in years 3 through 7?" If the answer requires real-time claim event streaming, sub-second accumulator queries for member-facing cost transparency, and configurable benefit matrices that ops can change without engineering, then the selection criteria for any vendor system is essentially: does this vendor provide an event-streaming API or am I committing to nightly batch exports forever? A vendor system purchased today on the basis of implementation speed that structurally prevents the data architecture you need in three years is not a $6M purchase — it is a $6M migration-in-waiting. Evaluate vendors on their data access model as aggressively as on their functional coverage.

MODULE 5 Regulatory & Compliance: What You're Actually Liable For ▶

Compliance is not a checkbox exercise for health insurance claims systems — it is structural. The regulatory surface area spans federal privacy law, ERISA preemption, ACA mandates, state-level prompt pay obligations, CMS requirements for government programs, and SOC 2 attestation for enterprise sales. Every engineer who touches claims data is touching regulated data. Every decision made by the claims adjudication engine

must be auditable. If your system cannot produce a complete, timestamped, decision-by-decision audit trail for a claim adjudicated three years ago, you are already non-compliant. This module will make you dangerous in a room with your compliance counsel — and dangerous enough to push back on bad architectural decisions made in the name of speed.

HIPAA: The Floor, Not the Ceiling

The Health Insurance Portability and Accountability Act of 1996 and its subsequent rules — the Privacy Rule (2003), Security Rule (2003), Breach Notification Rule (2009 via HITECH), and Omnibus Rule (2013) — define the baseline obligations for any entity that handles **Protected Health Information (PHI)**. PHI is any individually identifiable health information transmitted or maintained in any form: electronic (ePHI), paper, or oral. In a claims system, nearly everything is PHI: the 837 transaction, the EOB, the remittance advice, procedure codes tied to member IDs, diagnosis codes, provider NPI linked to a patient encounter.

The **Minimum Necessary Standard** is the most operationally underestimated HIPAA principle. Your systems must be designed so that users, processes, and integrations access only the PHI they need to perform their function. A customer service rep resolving a billing question should not have read access to clinical notes. Your data warehouse ETL jobs should not pull full unmasked SSNs when only the last four digits are needed for reporting. This is not aspirational — it is an architectural requirement that needs to be enforced at the data layer, not through policy documentation.

Business Associate Agreements (BAAs) are required with every vendor that handles PHI on your behalf. Your clearinghouse (Change Healthcare, Availity) requires a BAA. Your cloud provider (AWS, GCP, Azure) requires a BAA. Your analytics platform, your email vendor if it sends EOBs, your data warehouse, your logging infrastructure — all require BAAs. When Gravie or any TPA onboards a new vendor, the BAA execution should be a hard gate in the vendor approval process, not a retroactive checkbox. Engineers who spin up new SaaS tools that touch claims data without routing through legal for a BAA are creating reportable breach liability.

The **Breach Notification Rule** requires covered entities and business associates to notify affected individuals within 60 calendar days of discovering a breach of unsecured PHI.

For breaches affecting 500 or more individuals in a state, HHS must also be notified within 60 days and media outlets in affected states must be notified. The 60-day clock starts at discovery, not at confirmation of scope. In practice, this means your incident response runbooks must include a PHI impact assessment stage that completes within days, not weeks. Your logging and monitoring infrastructure must be able to identify which PHI records were exposed, by whom, and through what vector — because that is exactly what HHS will ask for during investigation.

The HIPAA Security Rule organizes ePHI safeguards into three categories. Administrative safeguards include workforce training, access management procedures, and incident response policies. Physical safeguards cover facility access controls, workstation security, and device/media controls — relevant even if your infrastructure is fully cloud-hosted, because it applies to developer laptops and office environments. Technical safeguards are the most architecturally significant: access controls (unique user IDs, automatic logoff, encryption), audit controls (hardware and software activity logs), integrity controls (preventing unauthorized alteration), and transmission security (encryption in transit). Note that the Security Rule does not mandate specific encryption standards — it requires "reasonable and appropriate" measures, which in 2024 means AES-256 at rest and TLS 1.2+ in transit is baseline, and anything weaker will fail an audit.

> 💡 **CTO Talking Point:** HIPAA does not prescribe technology — it prescribes outcomes. When your team debates whether to use field-level encryption vs. full-disk encryption vs. tokenization, the answer depends on your threat model, not on which is "HIPAA compliant." The right question to ask: "Can we demonstrate to an auditor that this control makes unauthorized access reasonably difficult and detectable?" Companies like Oscar Health and Clover Health have built compliance postures that treat HIPAA as a design constraint from day one — not a retrofit. The cost of retrofitting encryption or access controls onto a claims system that wasn't built for it is 10–30x the cost of building it right the first time.

ERISA: The Self-Funded Plan Framework

The Employee Retirement Income Security Act of 1974 governs employer-sponsored benefit plans, including self-funded health plans. Self-funded plans — the primary market for sophisticated TPAs like Gravie, Collective Health, and similar — are subject to ERISA rather than state insurance law in most respects. This is the most architecturally

significant regulatory fact about the self-funded market: ERISA preempts state insurance laws for self-funded plans, which means state-level coverage mandates that apply to fully-insured carriers do not automatically apply to self-funded plans. However, state prompt pay laws and state mental health parity laws have complex and evolving preemption status — do not assume full preemption without current legal guidance.

ERISA imposes fiduciary duties on plan administrators, including TPAs acting in fiduciary capacities. The duty of loyalty (acting in the interest of plan participants) and duty of prudence (using appropriate expertise and care) create legal exposure for claims processing decisions that appear arbitrary or financially motivated. Every claims denial must be supportable on clinical and contractual grounds — not just because it saves money. Your adjudication rules engine must be able to produce a written explanation for every denial decision that would survive a fiduciary challenge.

The Summary Plan Description (SPD) is the formal document that describes plan benefits to participants. ERISA requires SPDs to be written in plain language and distributed to participants. From a systems perspective, your claims adjudication logic must implement exactly what the SPD says — and when plan documents are updated (annually, typically), your adjudication configuration must update to match. Mismatches between the SPD and what the claims system actually pays are a fiduciary failure and a significant litigation risk. Collective Health has built substantial product capability around making plan document management and adjudication configuration synchronization tractable at scale.

Form 5500 is the annual reporting form that self-funded plans must file with the Department of Labor and IRS. Schedules within Form 5500 (particularly Schedule C for service provider compensation and Schedule H for financial data) require aggregated claims data that your system must be able to produce accurately. Employers rely on their TPAs to provide the data inputs for Form 5500 — if your reporting infrastructure cannot produce accurate aggregate claims and premium data by plan year, you create audit exposure for your employer clients.

ACA: Ongoing Mandates That Shape Claims Logic

The Affordable Care Act introduced a set of permanent mandates that are implemented as rules in claims adjudication systems. Essential Health Benefits (EHBs) define ten

categories of coverage that must be provided without annual or lifetime dollar limits in non-grandfathered individual and small group plans. The ten categories include: ambulatory patient services, emergency services, hospitalization, maternity and newborn care, mental health and substance use disorder services (at parity with medical/surgical benefits), prescription drugs, rehabilitative services, laboratory services, preventive and wellness services, and pediatric services including oral and vision.

Preventive care mandates require that USPSTF A/B recommendations, ACIP-recommended vaccines, and HRSA-supported women's health guidelines be covered at zero cost-sharing (no copay, no deductible) for in-network preventive care. This is implemented at the claims adjudication layer as a procedure code / service category check that bypasses normal cost-sharing logic. Post-Braidwood Management v. Becerra litigation has introduced uncertainty about the constitutional basis for specific preventive care mandates, but as of mid-2024 the mandates remain substantially intact — though compliance counsel should monitor ongoing litigation.

Out-of-pocket limits are annual caps on what members pay for in-network EHBs. For 2024, the limits are $9,450 for self-only coverage and $18,900 for family coverage. Embedded deductibles in family plans have specific interaction rules with these limits that are notoriously complex to implement correctly. The accumulator design in your claims adjudication system — how you track and apply member cost-sharing against the annual deductible and OOP max — is one of the highest-risk logic areas in claims processing. Errors in accumulator logic are common sources of member complaints, regulatory audits, and legal exposure.

The **No Surprises Act (NSA)**, effective January 2022, prohibits balance billing for emergency services and non-emergency services at in-network facilities from out-of-network providers. It also established an Independent Dispute Resolution (IDR) process for payment disputes between payors and providers. NSA implementation requires your claims system to identify qualifying surprise billing situations, apply the appropriate payment methodology (typically the Qualified Payment Amount, or QPA), and generate the required Explanation of Coverage documents. The NSA also requires **machine-readable files (MRFs)**: every group health plan must publish publicly accessible, machine-readable files containing in-network negotiated rates and out-of-network allowed amounts. These files must be updated monthly and are enormous — a large

national carrier's MRF can be multiple terabytes. Generating, hosting, and maintaining MRF compliance is a significant engineering and infrastructure problem.

State Regulations: The Long Tail of Compliance

While ERISA preempts many state insurance laws for self-funded plans, the preemption is not total — and for fully-insured plans, state regulation is comprehensive. TPAs operating nationally must maintain awareness of requirements in every state in which they operate. The most operationally significant state requirements are prompt pay laws, which mandate that insurers and sometimes TPAs pay clean claims within defined timeframes. Most states require payment within 30–45 days, with penalties ranging from interest on late payments to fines and license sanctions. Your claims SLA monitoring must track clean claim age against state-specific payment windows, and your workflow system must prioritize aging claims before they breach state deadlines.

State mandated benefits apply to fully-insured plans and vary substantially. Examples include autism spectrum disorder treatment mandates, infertility treatment mandates, diabetes supply coverage mandates, and mental health parity laws more expansive than the federal Mental Health Parity and Addiction Equity Act. If you operate in the fully-insured space — or administer plans for church or government employers that may not have full ERISA preemption — your adjudication rules engine must support state-specific benefit configuration by jurisdiction.

TPA licensing requirements vary by state. Approximately 40 states require TPAs to obtain a license before administering health benefit plans. License requirements typically include: financial reserves, key personnel background checks, fee disclosure, and filing of administrative service agreements. TPA license applications are not quick — plan 6–12 months for a new state. Your operational expansion strategy must sequence state licenses ahead of employer client launches in those states.

CMS Requirements: Medicare Advantage and Part D

If your claims system touches government programs — Medicare Advantage (MA) plans, Part D pharmacy benefit programs, or Medicaid managed care — CMS requirements layer on top of all of the above. Medicare Advantage plans are subject to CMS Star Ratings, which measure quality on dimensions including claims accuracy, timely appeals

processing, and member experience. Star Ratings directly affect plan revenue through quality bonus payments — a half-star improvement can mean hundreds of millions of dollars for a large MA plan. Claims processing accuracy is a direct input to Star Ratings, which means your SLA management and quality control processes have direct revenue impact in the MA space. Oscar Health's early focus on claims data quality and processing accuracy was directly connected to their MA Star Rating strategy.

Part D claims require integration with CMS's Prescription Drug Event (PDE) reporting system, which audits every pharmacy claim paid under the Part D benefit. PDE data reconciliation between your claims system and CMS's records is a recurring operational obligation with real financial reconciliation consequences. CMS performs Risk Adjustment Data Validation (RADV) audits of diagnosis coding submitted by MA plans — incorrect coding in your claims data that flows through to risk adjustment submissions creates retrospective payment recoupment risk.

Law / Framework	Who It Applies To	Key Claims System Obligation	Primary Enforcement Body	Pe
HIPAA Privacy Rule	Covered entities, BAs	Minimum necessary access, BAAs, PHI handling procedures	HHS OCR	$1 vic $1 ca
HIPAA Security Rule	Covered entities, BAs	Technical, physical, admin safeguards for ePHI	HHS OCR	Sa cri $2
HIPAA Breach Notification	Covered entities, BAs	60-day notice, media notice for 500+ in a state	HHS OCR	$1 vic
ERISA	Self-funded employer plans, TPAs as fiduciaries	Fiduciary-grade denial documentation, SPD accuracy, Form 5500 data	DOL, IRS	Ci $1 vic
ACA (EHB, preventive, OOP caps)	Non-grandfathered group and individual plans	Correct preventive care $0 cost-sharing logic, OOP accumulator accuracy	HHS, DOL, IRS (trifecta)	$1 fo co
No Surprises Act	Group health plans, carriers, providers	NSA claim identification, QPA calculation, MRF generation	HHS, DOL, Treasury, CMS	$1 vic pr co

Law / Framework	Who It Applies To	Key Claims System Obligation	Primary Enforcement Body	Pe
State Prompt Pay Laws	Carriers, some TPAs (state-specific)	Pay clean claims within 30–45 days	State DOI	Int lic
CMS / Medicare Advantage	MA plans, MA-PD plans	PDE reporting, RADV readiness, Star Rating quality measures	CMS	Cc te sc re
SOC 2 Type II	SaaS/technology service providers handling client data	Annual third-party audit of controls; required for enterprise sales	AICPA standards; customer contractual	Lo co re

SOC 2 Type II: The Enterprise Sales Gate

SOC 2 Type II is not a legal requirement — it is a market requirement. Any self-funded employer with 500+ employees, any health plan, and any sophisticated purchaser of TPA or claims processing services will require a current SOC 2 Type II report before signing a contract. The report, produced by an independent CPA firm, attests that a service organization's controls around five Trust Service Criteria (TSC) have been operating effectively over a defined period (typically 12 months). The five criteria are: Security (the foundation — access controls, encryption, incident response), Availability (system uptime and performance SLAs), Processing Integrity (completeness and accuracy of claims processing), Confidentiality (protection of confidential information beyond PHI), and Privacy (personal information handling consistent with the organization's privacy notice).

For a claims system, the Processing Integrity criterion is the most technically demanding. Auditors will sample claims and verify that the system processed them accurately, completely, and on schedule. They will test that your adjudication rules match your documented procedures, that exceptions are logged and resolved, and that your QA controls catch errors before payment. The gap between "we process claims accurately" and "we have documented, testable controls that prove we process claims accurately" is where most claims system companies underinvest until their first enterprise deal requires a SOC 2 report in 90 days.

SOC 2 Type II audits also scrutinize your vendor management program — the same BAA obligation chain required by HIPAA maps directly to your SOC 2 vendor inventory. Your cloud infrastructure, your database providers, your monitoring tools, your deployment pipelines all fall within scope. Infrastructure-as-code approaches (Terraform, CDK) and automated compliance tooling (Vanta, Drata, Secureframe) are now standard in the health tech space and significantly reduce the overhead of maintaining SOC 2 controls continuously rather than scrambling before each annual audit period.

Audit Trails: Every Decision Must Be Defensible

The most important architectural requirement for a claims system is one that rarely appears in functional specifications: every claim decision must generate a permanent, tamper-evident, human-readable audit record. This is not just about HIPAA or SOC 2 — it is about legal defensibility, fiduciary accountability, and operational quality control. When a provider disputes a claim payment three years after adjudication, or when a DOL audit requests documentation of plan administration decisions, or when a large employer client terminates their contract and demands their claims data, your audit trail is what makes the difference between a manageable situation and catastrophic exposure.

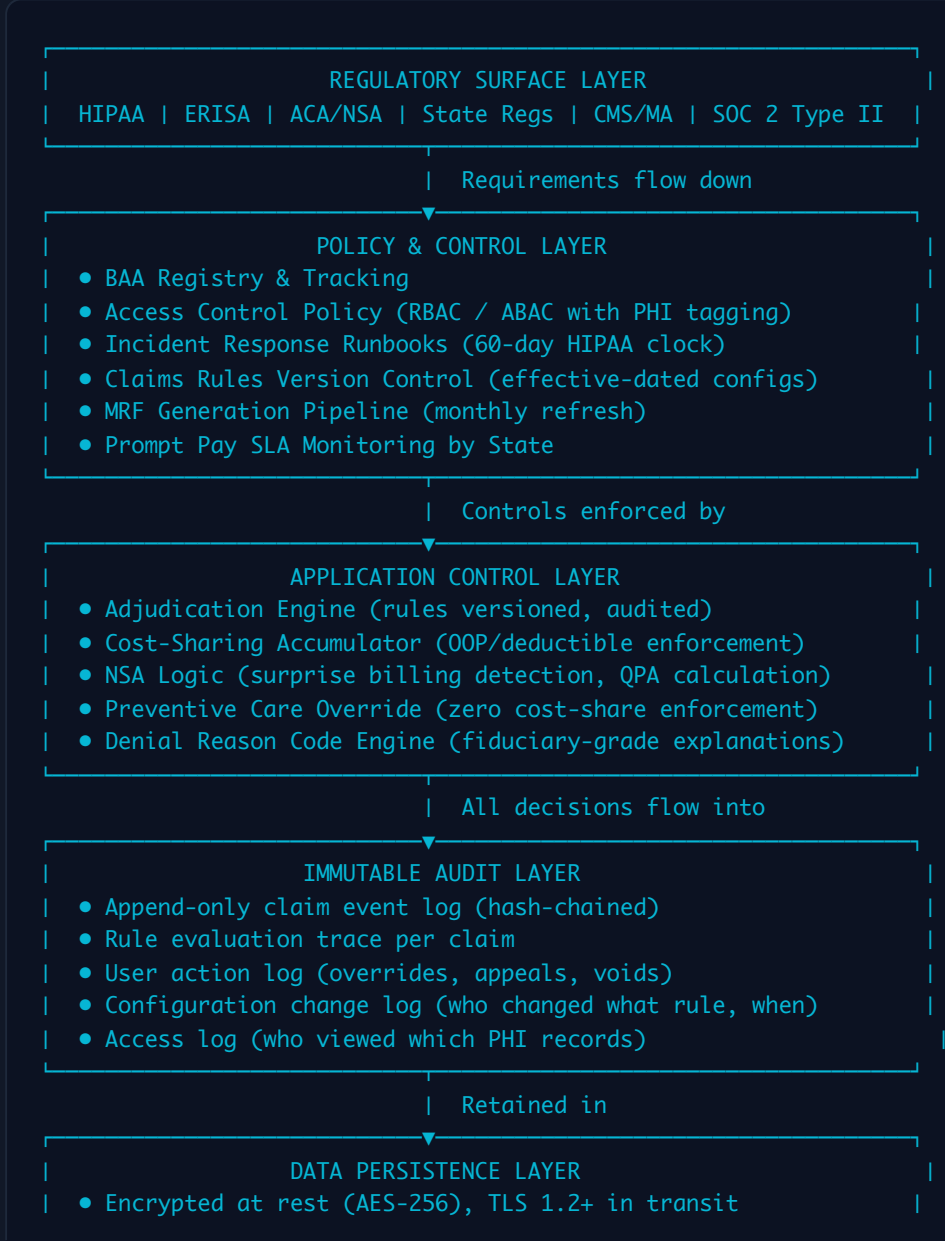
What auditors look for in a claims audit trail: the original claim as received (date, time, submitter, transaction ID); each rule evaluated during adjudication (rule name, version, input values, output decision); the adjudicator identity (system process or human user, with credential); any edits or overrides made to the claim after initial processing (who, when, what changed, reason code); the payment calculation including each cost-sharing step; the EOB generation event; and any subsequent adjustments, appeals, or voids with their own complete decision trails. The audit trail must be immutable — records cannot be edited or deleted. Append-only logging with cryptographic integrity verification (hash chaining, write-once storage) is the appropriate architecture.

Data retention requirements for claims data are driven by multiple overlapping obligations. ERISA requires plan records to be retained for six years from the date of filing or the date when records were required to be filed. State insurance regulations typically require 7–10 years. Medicare claims records require retention for 10 years. HIPAA requires PHI to be retained for six years from creation or last use, whichever is later. In practice, the defensible position is 10 years for all claims records, with audit logs retained for the

same period. Storage is cheap; litigation is not. Design your data lifecycle and archival strategy for 10-year retention from day one.

> 💡 **CTO Talking Point:** The question CTOs consistently underestimate is: "Can we reconstruct the exact state of our adjudication rules engine as it existed on any given date in the past?" Claim disputes and audits often require you to re-adjudicate a claim under the rules that were in effect when the original decision was made — not today's rules. This requires versioning your adjudication configuration, not just your application code. Treat your claims rules engine configuration as immutable, versioned artifacts with effective date ranges. Companies like Collective Health and Gravie that handle large employer accounts with custom plan designs have learned this lesson either proactively or expensively.

Compliance Architecture: The Layered Model



```
|  • Write-once audit storage (S3 Object Lock / WORM)            |
|  • 10-year retention lifecycle policy                          |
|  • Geographic data residency controls (state-specific)         |
|  • Backup with point-in-time recovery (PITR 35+ days)          |
└────────────────────────────────────────────────────────────────┘
```

This layered model reflects the principle that compliance is not a feature — it is a cross-cutting concern that must be present at every tier of the architecture. The regulatory surface layer changes over time (new rules, new enforcement priorities) and requirements flow down through policy into application logic and ultimately into data. The immutable audit layer is the one layer that can never be optional or deferred. A claims system that is processing correctly but leaving no traceable audit record is a liability waiting to materialize.

For Gravie specifically, the compliance architecture must accommodate a hybrid model: fully-insured plans (subject to state insurance regulation), self-funded plans (ERISA-governed, state insurance law largely preempted), and potentially individual market or marketplace plans (full ACA applicability). The adjudication configuration system must support plan-level regulatory tagging so that the correct rules are applied based on plan type, not based on a single uniform configuration. This is one of the more complex multi-tenancy problems in health insurance claims: the same underlying claims processing logic must produce different regulatory outcomes depending on which employer's plan the member belongs to.

MODULE 6 Integration Architecture: The Claims System Is a Hub, Not an Island ▶

One of the most common architectural mistakes made by engineers entering the health insurance space is treating the claims system as the primary system — the thing everything else connects to. The reality is more chaotic: the claims system is one critical node in a web of bilateral integrations with clearinghouses, pharmacy benefit managers, enrollment systems, banking infrastructure, stop-loss carriers, provider networks, and a growing constellation of analytics and member-facing products. The integrations are not a secondary concern to be handled after the core claims logic is built. They are the claims system. The adjudication engine is worthless if it cannot reliably receive claims, verify eligibility, route to the correct benefit configuration, and generate remittances — all of which require working integrations. This module maps that full integration surface and

explains the architectural choices that determine whether your system can evolve or will calcify.

The Full Integration Map

Clearinghouses are the first integration that must work. The dominant players are Change Healthcare (now Optum, post-UnitedHealth acquisition) and Availity. Clearinghouses serve as the interchange layer between providers submitting 837 transactions and payors receiving them, performing format validation, duplicate detection, trading partner routing, and 277 acknowledgment generation. They also route 835 Electronic Remittance Advice files back to providers. The practical dependency on Change Healthcare became viscerally clear in February 2024 when a ransomware attack on Change Healthcare's systems caused a multi-week outage that disrupted claims flow for thousands of providers and payors nationwide. Any claims system architect who had assumed clearinghouse availability as a given now has a compelling case study for why clearinghouse redundancy — maintaining concurrent connectivity with at least two clearinghouses and the capability to route around a primary clearinghouse failure — is a resilience requirement, not an optimization.

Provider directories and network management are the source of truth for which providers are in-network, what their contracted rates are, and what specialties and locations they serve. Your claims adjudication depends on accurate, current provider directory data: is this rendering provider in-network for this member's plan? What is the contracted rate for this procedure at this facility? The provider directory integration is bidirectional — your system needs to receive directory updates (provider credentialing, terminations, rate changes) and must surface directory data to member-facing applications and eligibility verification. Directory data quality is chronically poor across the industry; the No Surprises Act has imposed additional data accuracy requirements that have created legal exposure for inaccurate directories. Investing in directory data governance — primary source verification, provider self-service attestation portals, automated discrepancy detection — has measurable ROI in both compliance risk reduction and claims accuracy.

PBM integration handles pharmacy claims, which are fundamentally different from medical claims in their processing model. Pharmacy claims are adjudicated in real time at the point of sale (the pharmacy), not retrospectively after the service is rendered. Your

PBM partner (Express Scripts, CVS Caremark, OptumRx for large volumes; Navitus, Capital Rx, RxBenefits for more transparent alternatives) operates its own real-time adjudication system. Your integration requires a formulary data feed for member benefit display, coordination of benefits logic that matches medical and pharmacy accumulator data, and 835-equivalent remittance data from the PBM for your claims records. Gravie's approach to pharmacy — and the broader employer-facing TPA space's move toward transparent PBM models — reflects recognition that pharmacy is 20–30% of employer health spend and historically opaque. The integration architecture must support comparing PBM pass-through pricing data against claims paid to verify contract performance.

Banking and ACH integration is how money moves. Provider payments via ACH require your system to generate NACHA-formatted ACH files from approved remittance data, submit them to your banking partner, and reconcile returned files (rejections, corrections, prenotes) against your payment records. Member reimbursement for direct member-pay claims (HSA reimbursements, out-of-network balance submissions) uses the same ACH infrastructure with a different payment direction. The reconciliation loop — matching ACH settlement data back to individual claim payments — is a critical control that prevents both overpayments and underpayments from going undetected. Automated daily reconciliation with exception flagging is table stakes; manual reconciliation at any significant claims volume is a control failure waiting to be discovered in an audit.

Stop-loss carrier feeds are required for any self-funded plan that carries stop-loss insurance (which is nearly all of them). Specific stop-loss coverage caps the plan's liability for any individual member's claims above a threshold (typically $50K–$250K). Aggregate stop-loss caps total plan liability. Stop-loss carriers require monthly or quarterly claims data feeds in their proprietary formats to track accumulations toward individual deductible thresholds. Timely, accurate stop-loss feeds directly affect the employer's ability to recover against catastrophic claims — missed or late feeds can jeopardize reimbursement. Your claims system must generate and deliver stop-loss feeds as a first-class operational process, not an afterthought report.

Enrollment system integration via 834 transactions is the eligibility foundation. The ASC X12 834 Benefit Enrollment and Maintenance transaction carries member enrollment, termination, and change events from HR and benefits administration platforms (Workday, ADP, BambooHR, Benefitfocus, isolved) to your claims system. 834

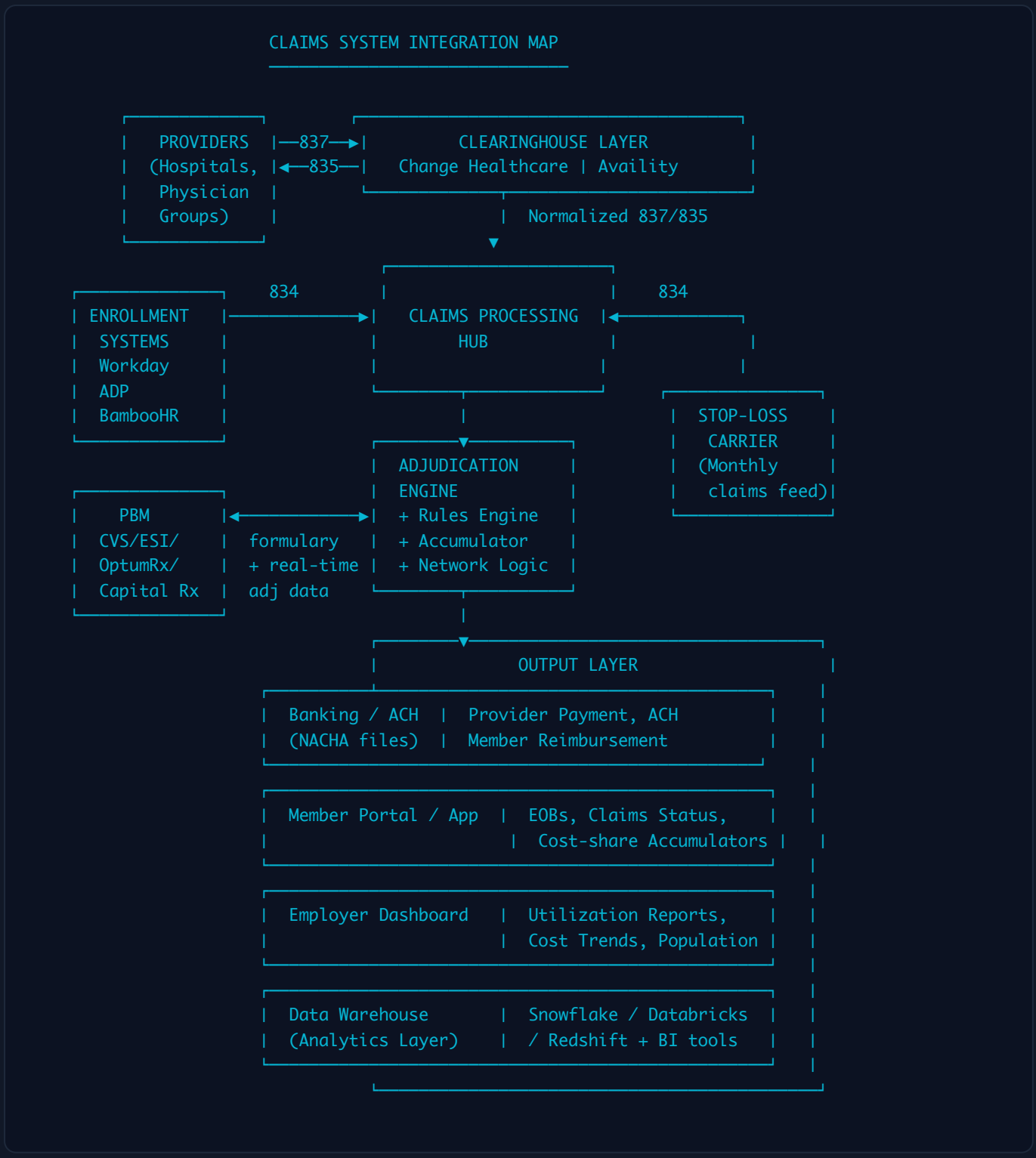
processing is notoriously complex: the transaction format supports dozens of loop/segment combinations for different employment situations, and HR platforms emit 834s with idiosyncratic interpretations of the standard. Your 834 processing must be robust to malformed segments, handle retroactive enrollment and termination (and the claims adjustments that follow), and surface enrollment exceptions for human resolution rather than silently dropping records. Eligibility data that is stale or incorrect causes member-facing failures — members presenting at providers who are told they are not covered — which are among the highest-impact member experience failures and generate the most regulatory complaints.

Member-facing portal and mobile app integration surfaces claims status, EOBs, cost-sharing accumulators, provider search, and digital ID cards to members. This integration requires a well-defined internal API layer — the same API that your member portal uses should be the same API your employer dashboard uses, with appropriate scoping. Treating the member portal as a bespoke integration that reads directly from the claims database creates a fragile coupling that makes the claims system harder to evolve. Oscar Health built significant competitive differentiation on their member-facing product, which required investing in a clean internal API layer over their claims data — not because the API was a compliance requirement, but because it was the only way to iterate on member experience without breaking claims processing.

Employer reporting dashboard surfaces aggregate claims analytics, utilization trends, cost drivers, and population health metrics to HR and finance teams. This is a read-only integration, but it is commercially critical — employers who cannot see their claims data in near-real-time will not renew their TPA contract. The reporting dashboard should be fed from a separate analytical replica or data warehouse — never from production claims data — to avoid reporting queries degrading transactional performance.

Data warehouse and analytics platform integration closes the loop on claims intelligence. Snowflake, Databricks, and Amazon Redshift are the dominant platforms in the employer health space. Claims data must be continuously synchronized to the warehouse (typically via CDC from the operational database or via an event stream) to support actuarial analysis, predictive modeling, member risk stratification, and care management program identification. The claims system is the data source; the warehouse is where that data becomes actionable. Clover Health built much of their clinical value proposition on the quality and freshness of their claims-to-analytics pipeline

— their clinical operations teams needed near-real-time claims signals to identify members for proactive outreach.



API-First vs EDI Batch: The Modernization Tension

The health insurance industry was built on EDI batch processing — X12 transactions submitted in bulk files, processed overnight, and reconciled the next business day. This model made sense when network bandwidth was expensive, computing was centralized,

and providers sent claims in paper batches that were then keypunched. It makes much less sense in 2024, when members expect real-time eligibility verification, providers want immediate claim status, and employer clients want same-day utilization data. The tension between EDI batch and API-first architectures defines the modernization challenge for most claims systems.

Traditional EDI batch has real advantages that are easy to undervalue: it is standardized (X12 is a federal standard, not proprietary), battle-tested at enormous scale, and universally supported by clearinghouses and major providers. Every hospital system in the country can submit an 837. Not every hospital system can call a JSON REST API. The economic reality is that EDI batch will not disappear for large-volume provider transactions for at least a decade — the network effects of the X12 standard are too deep.

API-first architecture enables the real-time experiences that modern health products require. Real-time eligibility verification (271 response in <500ms), instant claim status, dynamic benefit display, and real-time prior authorization are all API-native use cases. The CAQH CORE operating rules have driven real-time eligibility APIs into mainstream adoption; major clearinghouses and BCBS plans now offer real-time 270/271 APIs. The question is not whether to build API-first — you must — but whether you can maintain both the batch EDI pipeline and the real-time API surface simultaneously without fragmenting your data model.

FHIR (Fast Healthcare Interoperability Resources) is the emerging standard that bridges this divide. FHIR R4, published by HL7, defines a RESTful API standard with a JSON data model for health information exchange. The CMS Interoperability and Patient Access Rule (2020) requires CMS-regulated payors to implement FHIR APIs for member data access. The CMS Prior Authorization Rule (2024) requires FHIR APIs for prior authorization workflows by 2026–2027 for most plan types. FHIR is not optional for any claims system that intends to remain compliant with CMS rules.

The key FHIR resources relevant to claims processing are: Patient (member demographics, unique identifiers), Coverage (insurance eligibility and benefit information), Claim (the claim submission resource — the FHIR equivalent of the 837), and ExplanationOfBenefit (EOB) (the FHIR equivalent of the 835 — the adjudicated claim with cost-sharing details). The EOB resource is the most data-rich: it carries the full claim context, every line item, the adjudication decision, cost-sharing applied, and payment

amounts. The CARIN Blue Button Implementation Guide defines a consumer-directed FHIR EOB profile used for member data access. The Da Vinci Project defines FHIR implementation guides for prior authorization (PAS), payer-to-payer data exchange (PDex), and formulary (Drug Formulary IG) — all of which your system will need to implement to remain competitive and compliant.

Dimension	EDI Batch (X12)	REST API (Proprietary)	FHIR R4
Latency	Hours to overnight	Real-time (<1s)	Real-time (<1s)
Standardization	Federal standard (X12 HIPAA)	Proprietary per vendor	HL7 standard; CMS-mandated for regulated plans
Provider adoption	Universal — every provider system	Variable; larger systems only	Growing; EHR vendors (Epic, Cerner) support it
Use cases	Claims submission (837), remittance (835), eligibility (270/271), enrollment (834)	Real-time eligibility, claim status, prior auth, member data access	Patient data access, prior authorization (PAS), payer-to-payer (PDex), formulary
Regulatory driver	HIPAA transaction standards (required)	None — market-driven	CMS Interoperability Rule (required for regulated plans); expanding via DA Vinci
Data model	Segment/loop structure; flat, verbose	JSON/XML; varies by vendor	Structured JSON with typed resources; rich relationships
Error handling	997/999 acknowledgment files; asynchronous	HTTP status codes; synchronous	OperationOutcome resource; synchronous or async
Versioning	X12 version-specific (5010 current)	Custom versioning strategy	FHIR R4 / R5 with explicit versioning support
When to use	All large-volume provider transactions; enrollment batch processing	Internal system APIs; legacy partner integrations	Member-facing data access; prior auth; payer-to-payer; new regulatory mandates

Real-Time Eligibility vs Batch Enrollment

Eligibility verification happens at two different points in time with different technical requirements. Batch enrollment via 834 transactions handles the baseline: new hires enrolled, terminations processed, qualifying life events applied. This is your master record of who is covered on which plan with which benefit tier. Batch 834 processing must be idempotent — reprocessing the same 834 file should produce the same eligibility state, not duplicate records. Effective dating is critical: a member terminated on the 15th of a month has different claims adjudication rights for claims dated on the 14th vs. the 16th, and your accumulator logic must handle retroactive terminations and their downstream claims adjustments.

Real-time eligibility via 270/271 transactions serves a different purpose: it answers the question "Is this person eligible right now?" for a provider at the point of care. Real-time eligibility checks must return a response within seconds — CMS CORE operating rules require sub-20-second response times, and major clearinghouses now routinely return responses in under 500ms. Your real-time eligibility endpoint must be available at five-nines uptime during business hours, because a provider's inability to verify eligibility delays care delivery and creates denials-avoidance problems. Real-time eligibility is architecturally separate from your batch enrollment processing — it should read from a highly-available eligibility cache or read replica, not directly from your primary claims database.

The tension between batch and real-time eligibility creates a temporal consistency challenge: if a member's enrollment is updated via a batch 834 at 2 AM, your real-time eligibility endpoint must reflect that update by 8 AM when providers start calling. If there is a batch processing failure, your eligibility cache becomes stale. You need monitoring that detects eligibility staleness and alerts before it causes member-facing failures. The 834 processing pipeline is not a background job — it is a critical path that must have SLAs, monitoring, and on-call escalation.

Event-Driven Architecture for Claims Processing

Traditional claims systems are built around a synchronous request-response model: receive an 837, run adjudication, write a database record, generate an 835. This model is simple and debuggable, but it creates tight coupling between processing stages, makes it difficult to add new downstream consumers (analytics, fraud detection, member notifications), and makes it hard to scale specific bottlenecks independently. The

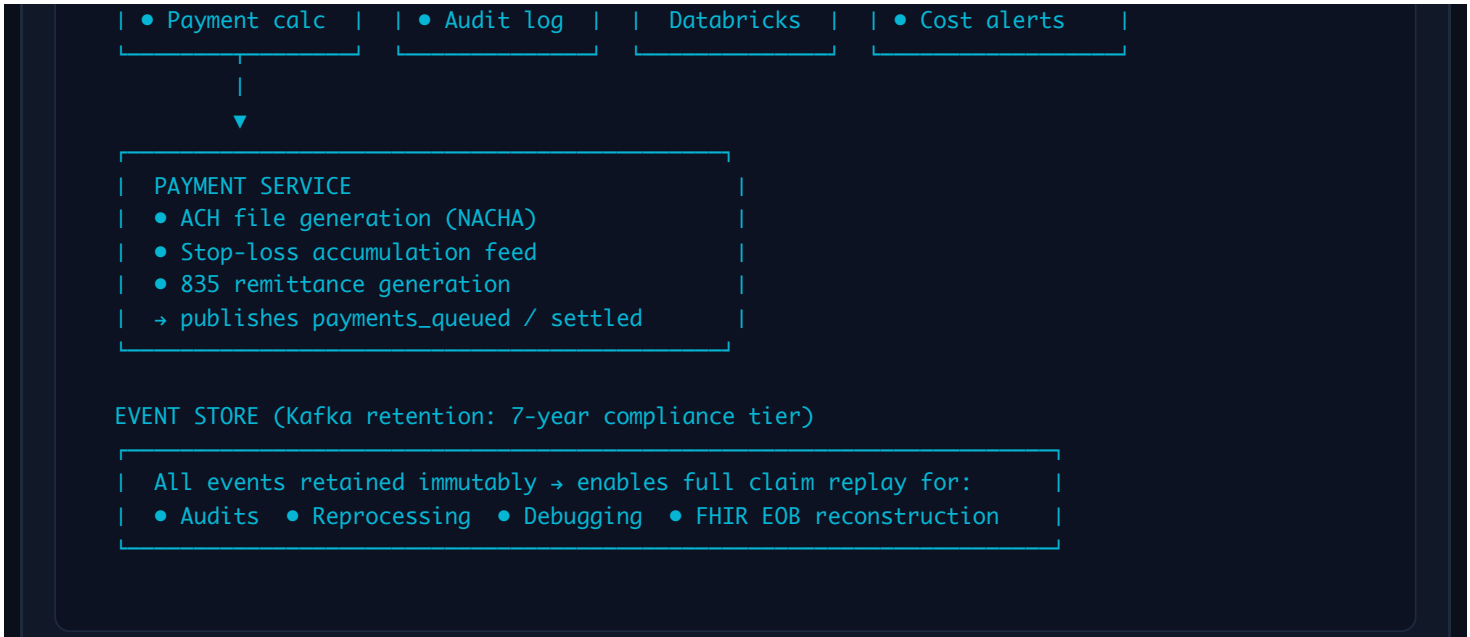
architectural pattern that has replaced it in modern health tech systems is event-driven architecture using a persistent event log.

Apache Kafka has become the dominant event streaming platform in this space, used by Oscar Health, Clover Health, and Collective Health among others for claims event pipelines. The core insight is: every state change in a claim is an event. Claim received. Claim validated. Eligibility verified. Claim queued for adjudication. Adjudication started. Rule evaluated. Adjudication completed. Payment calculated. EOB generated. ACH payment queued. Payment settled. Each of these is a distinct, immutable event that can be written to a Kafka topic, consumed by multiple downstream systems, and replayed for reprocessing, debugging, or audit purposes.

Event sourcing is the pattern where the event log is the system of record — the current state of a claim is derived by replaying all events for that claim, not by reading a current-state record. This is architecturally powerful for claims because it means your audit trail is your data model, not an append to it. Every adjudication decision, every override, every correction is a first-class event. Replaying events lets you answer questions like "What would this claim have paid under last year's rules?" or "Show me every claim that was affected by the rate change on March 15th." The downside is that event sourcing adds complexity: CQRS (Command Query Responsibility Segregation) is the typical companion pattern, where write operations go through the event log while read operations use projected read models (Postgres, Elasticsearch, Redis) that are updated by consuming the event stream.

For claims processing specifically, the Kafka topics that matter are: raw_claims_received (837 transactions as received, before any parsing), claims_validated (post-format-validation, pre-adjudication), claims_adjudicated (post-adjudication with full decision detail), payments_queued (approved payments ready for ACH generation), payments_settled (ACH confirmation from banking), and claims_adjusted (any post-adjudication corrections). Downstream consumers of these topics include: the analytics pipeline (claims_adjudicated → Snowflake/Databricks via Kafka Connect), the fraud detection service (claims_validated → ML scoring service), the member notification service (claims_adjudicated → push/email notification), and the stop-loss feed generator (claims_adjudicated → stop-loss accumulation tracker).

💡 **CTO Talking Point:** The most underestimated operational benefit of event-driven claims architecture is reprocessing. In a traditional synchronous claims system, if you discover that an adjudication rule was misconfigured for a three-month period, reprocessing the affected claims requires identifying them, reversing payments, re-adjudicating, and generating new remittances — all while maintaining the audit trail of what happened. In an event-sourced system, reprocessing is a first-class operation: replay the raw_claims_received events for the affected period through the corrected rules engine and project the new state. Collective Health built their platform with this reprocessing capability explicitly in mind, which has allowed them to correct plan configuration errors without the data integrity nightmares that plague legacy claims systems. The upfront architectural investment pays for itself the first time you need to reprocess a large claim cohort.



Kafka Configuration and Operational Considerations

Operating Kafka for a HIPAA-regulated workload requires specific configuration choices. Kafka topics containing PHI must have encryption at rest enabled (KMS-encrypted at the broker or via client-side encryption). Consumer group offsets must be managed carefully — a misconfigured consumer that loses its offset could cause a claims processing service to re-process events already acted upon, creating duplicate payments. Claims processing consumers must be idempotent: processing the same event twice must produce the same result. This typically means using event IDs as idempotency keys and maintaining a processed-event store in Redis or Postgres before writing any side effects.

Topic retention for HIPAA compliance drives infrastructure sizing decisions. The 7–10 year data retention requirement applies to claim events as much as to database records. Kafka's native retention is typically configured in days or weeks for performance; for compliance, claims events must be archived to long-term storage (S3 with WORM / Glacier) before Kafka purges them. A common pattern is a dedicated compliance consumer that writes all claim-related events to S3 object storage with server-side encryption and object lock enabled — creating a compliance archive that is independent of Kafka's retention policy.

Schema evolution is a persistent challenge in event-driven claims systems. The FHIR R4 Claim resource schema, the proprietary adjudication event schema, and the X12 837 parsing output schema all change over time. Apache Avro with a Schema Registry (Confluent Schema Registry or AWS Glue Schema Registry) is the standard approach for

managing backward-compatible schema evolution in Kafka-based systems. Enforcing schema compatibility rules at the registry level prevents producers from publishing breaking changes that would corrupt downstream consumers.

> 💡 **CTO Talking Point:** The clearinghouse concentration risk exposed by the February 2024 Change Healthcare outage is a forcing function for architectural resilience that most claims systems had not invested in. The right response is not just adding a backup clearinghouse — it is designing your clearinghouse integration layer as an abstraction that routes claims based on availability, cost, and SLA. Your ingestion service should speak to both Change Healthcare and Availity via a routing layer that can shift volume in real time based on availability signals. This is a pattern that Oscar Health and other modern carriers have implemented. The cost of maintaining dual clearinghouse connectivity is a few engineer-weeks of work and modest per-transaction fees on the secondary clearinghouse — trivially small compared to the revenue impact of a multi-week claims processing outage. Any claims architecture review should now include explicit resilience testing against primary clearinghouse unavailability.

The Path Forward: FHIR-Native Architecture

The end state that the industry is moving toward — driven by CMS mandates, payer-to-payer data exchange requirements, and the computational advantages of FHIR's structured data model — is a FHIR-native claims system where FHIR resources are the primary data model, EDI is a translation layer at the edges (for backward compatibility with the installed base of legacy providers), and FHIR APIs are the primary integration surface for new integrations. In this model, an 837 received from a clearinghouse is parsed and transformed into a FHIR Claim resource at the ingestion boundary. Adjudication operates on FHIR resources. The output is a FHIR ExplanationOfBenefit. The 835 remittance is generated by transforming the FHIR EOB at the output boundary.

This architecture has three compounding benefits. First, compliance: FHIR EOBs are directly compliant with the CMS Patient Access API requirements — you do not need a separate transformation layer to serve member data access requests. Second, interoperability: when a member switches health plans, their claims history can be transferred via the FHIR PDex payer-to-payer exchange standard without any custom integration work. Third, analytics: FHIR resources are structured JSON with typed fields and code system references (ICD-10, CPT, SNOMED), which makes analytics and ML

modeling substantially more tractable than working with the flat, segment-based structure of X12 transactions.

The migration from legacy EDI-centric architecture to FHIR-native is a multi-year effort for any established claims system. The practical strategy is to introduce FHIR as the internal canonical model for new development — all new services consume and produce FHIR resources — while maintaining the EDI processing pipeline as a translation layer that feeds the FHIR model. Over time, the EDI layer becomes thinner as more integrations move to FHIR-native. Gravie, as a relatively modern TPA, has an architectural advantage over legacy carriers in this migration: the absence of decades of COBOL-based batch processing infrastructure means the migration path is shorter and the cultural resistance to change is lower.

MODULE 7 **Analytics, AI, and Fraud** ▶

The claims data asset is one of the most underexploited moats in health insurance. Every adjudicated claim is a structured event: who received care, from whom, at what cost, under what diagnosis. At scale, this data enables predictive modeling, fraud detection, network optimization, and population health management. Most legacy payers treat their data warehouse as a reporting afterthought. Companies like Oscar Health and Clover Health have built their competitive advantage almost entirely on treating claims data as a first-class product. This module covers how to build that capability — and where to buy versus build.

Claims Analytics: The Core Metrics

Before you can do ML, you need clean longitudinal claims data and a competent analytics layer. The foundational metrics every health-tech CTO must understand are not abstract — they are the lingua franca of every board meeting, actuarial review, and vendor negotiation.

PMPM (Per Member Per Month) is the atomic unit of health plan economics. Total medical spend divided by member-months. When your CFO says "our PMPM is $420," they mean the plan spent $420 per enrolled member per month on average. Your analytics platform must be able to slice PMPM by service line (inpatient, outpatient,

pharmacy, behavioral), by employer group, by geography, and by risk tier. A claims system that cannot produce PMPM in real time is analytically blind.

Medical Loss Ratio (MLR) is medical spend divided by premium revenue. Under the ACA, large-group plans must maintain MLR above 85% or issue rebates. If your MLR is 92%, you have thin administrative margin. If it's 78%, you're over-pricing or under-serving. MLR is a board-level metric that your claims system directly influences — faster adjudication, more accurate auto-adjudication, and fraud prevention all drive MLR improvement.

Utilization rates measure how frequently members use services: inpatient admissions per 1,000 members, ER visits per 1,000, generic drug dispensing rates. These metrics are the early warning system for population health trends. A 15% spike in ER utilization in a specific employer group is an actionable signal — it may indicate a mental health crisis, a primary care access gap, or a high-cost claimant cohort entering the plan.

High-cost claimant identification is disproportionately important: in most commercial populations, 1% of members drive 25-30% of medical spend. These are members with cancer, ESRD, complex neonates, or multi-system chronic disease. Identifying them early — before they've exhausted their deductible and before a catastrophic admission — and routing them into case management programs can produce $50,000-$200,000 in avoided costs per member. Your analytics pipeline must flag these members within 30 days of plan enrollment, not at the end of the year.

Benchmarking against external data sources (Truven MarketScan, IBM Watson Health, Milliman benchmarks) lets you answer: "Is our population sicker or healthier than a comparable commercial population?" Without this context, your PMPM is a number without meaning. Gravie's open enrollment model — members choose their own plan each year — creates unusual adverse selection dynamics that make benchmarking against standard commercial populations misleading. Your data team needs to understand these quirks.

Predictive Modeling: Risk Stratification and Care Management

Risk stratification is the practice of scoring each member by expected future cost. The dominant model is the Hierarchical Condition Category (HCC) model, originally developed for CMS Medicare Advantage risk adjustment, but widely adopted in commercial contexts. HCC assigns each member a risk score based on their documented

diagnoses: a member with diabetes + CKD + CHF might have an HCC score of 2.4, meaning their expected costs are 2.4x the average. Critically, HCC scores are based on diagnoses from claims — so incomplete coding by providers directly suppresses your risk scores and your reimbursement in Medicare Advantage contexts.

Predictive models for care management intervention go beyond HCC. You want to identify members who are high-risk but not yet high-cost — the pre-catastrophic cohort. Features for these models include: emergency department utilization velocity, medication adherence gaps (inferred from pharmacy claims), gap in preventive care (no PCP visit in 18 months), and diagnosis trajectory (new HCC codes appearing in the last 90 days). Oscar Health's care team model is built on exactly this kind of signal — identify the member before the hospitalization, not after.

> 💡 CTO Talking Point: Risk stratification is not just a clinical tool — it's a data quality audit. If your HCC scores are systematically lower than actuarial expectations, your providers are undercoding. That means your risk-adjusted reimbursement (in Medicare Advantage) is too low, and your care management program is missing high-risk members. The fix is clinical documentation improvement (CDI) programs, which require your data team to surface the gap to the provider network. This is a $10-50 PMPM problem at scale.

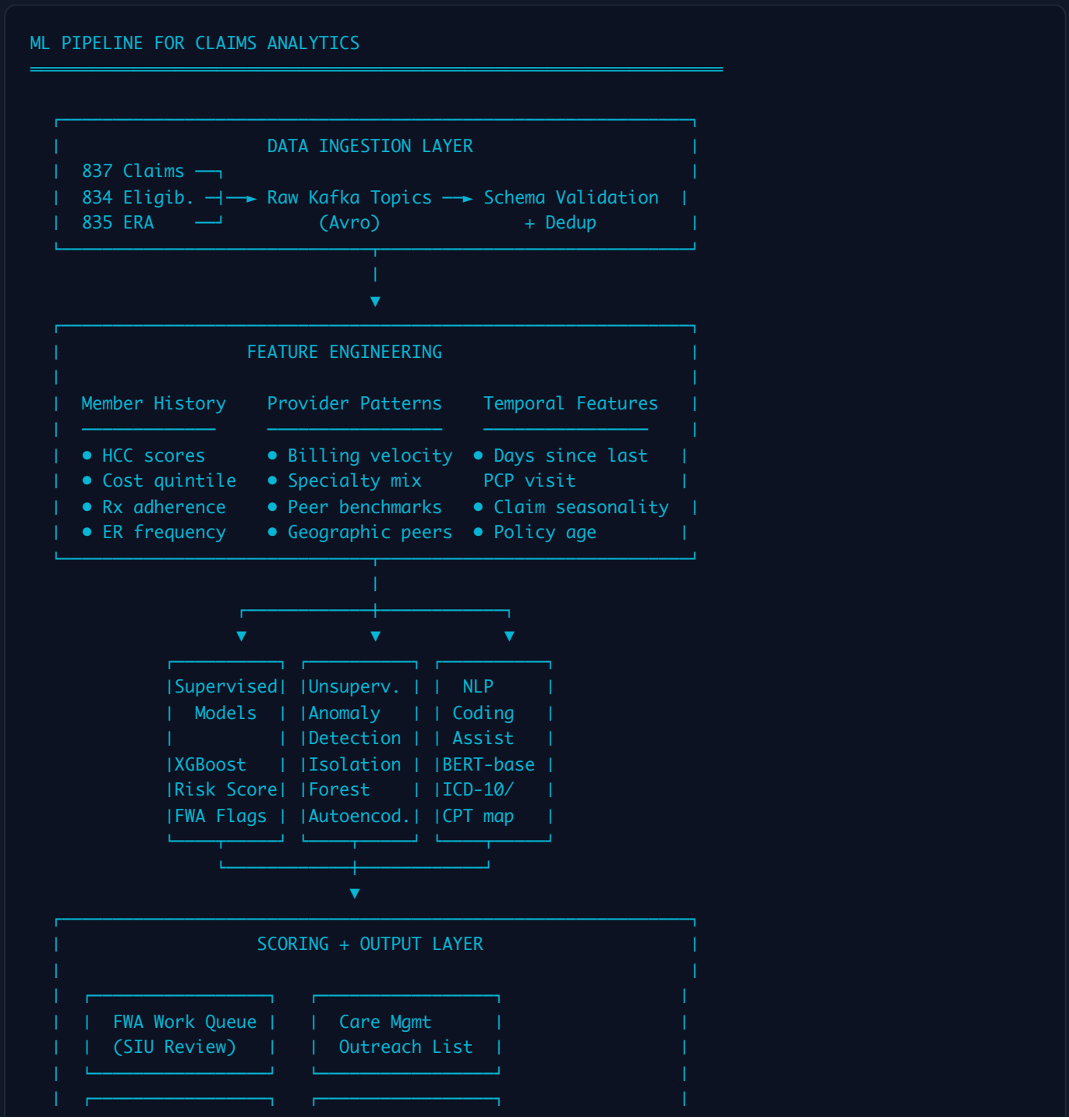
AI in Claims: Where It Actually Works

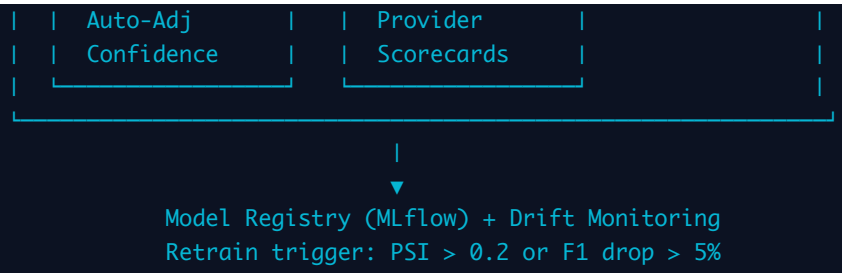
Auto-coding using NLP is one of the most commercially mature AI applications in healthcare. Clinical notes — discharge summaries, operative reports, ED visit documentation — arrive as unstructured text. NLP models extract ICD-10 diagnosis codes and CPT procedure codes from this text, reducing manual coding effort and improving coding accuracy. Companies like Optum360, 3M, and Nuance (now Microsoft) have sold these tools for a decade. For a CTO at a health-tech startup, the build-vs-buy answer here is nearly always buy: the NLP models require massive labeled training datasets (hundreds of thousands of coded clinical notes) and deep medical coding expertise to validate. You will not beat Nuance on clinical NLP with a 10-person ML team.

Clinical NLP for unstructured data extends beyond coding. Prior authorization workflows benefit from NLP that extracts clinical criteria from submitted medical records. Grievance and appeal letters can be automatically triaged by clinical issue type. Lab report

interpretation for high-cost condition identification is another use case. These applications are more tractable for internal development because the labeled dataset is your own operational data, not general clinical text.

Anomaly detection for claim patterns is where ML pays significant dividends on proprietary data. Rather than rules ("flag all claims over $10,000"), anomaly detection models learn the normal distribution of claim patterns for each provider, service type, and member cohort — and flag statistically unusual deviations. A dermatologist who bills 60 units of Mohs surgery per day is unusual. A DME supplier whose claim velocity tripled in Q3 is unusual. These patterns are invisible to rules-based systems because the thresholds are inherently relative, not absolute.





Fraud, Waste, and Abuse (FWA): Taxonomy and Detection

FWA represents an estimated 3-10% of total healthcare spending — roughly $300 billion annually in the US. The distinction matters: fraud is intentional misrepresentation, waste is unnecessary services or inefficient use of resources (often not criminal), and abuse is practices that are inconsistent with sound fiscal, business, or medical practices. Your detection systems need to handle all three, but legal escalation paths differ significantly.

FWA Type	Description	Detection Signal	ML Approach	Real-World Example
Duplicate Billing	Same service billed multiple times, sometimes with minor field variation to evade exact-match dedup	Fuzzy matching on DOS, provider, member, service code; slight date offsets; split billing across TINs	Entity resolution / record linkage models; Levenshtein distance clustering	Home health agency bills same visit under two NPI numbers for same member on same day
Upcoding	Billing a higher-complexity E&M code (99215) when documentation supports a lower one (99213)	Provider's E&M code distribution vs. specialty peer group; outlier rate in high-complexity codes	Supervised classification on provider coding patterns; peer benchmarking regression	Primary care physician bills 99215 for 90% of visits when specialty average is 35%
Unbundling	Billing component procedure codes separately when a bundled code (and lower reimbursement) applies	NCCI edits violations; co-occurrence of codes that should be bundled per CMS logic	Rules engine + anomaly detection on code-pair frequencies; CCI edit compliance scoring	Surgeon bills CPT 27447 (total knee) plus component codes that are included in the global surgical package
Phantom Billing	Billing for services never rendered; fabricated patient	Member denies receiving service on audit; provider location vs. member	Cross-claim correlation (member was inpatient at	DME company bills for power wheelchairs for members who have no mobility diagnosis and

FWA Type	Description	Detection Signal	ML Approach	Real-World Example
	encounters or procedures	location mismatch; service during member hospitalization elsewhere	Hospital A when provider B billed outpatient visit); geospatial anomaly detection	live in walkable urban areas
Provider Collusion Rings	Networks of providers (often including recruiters who pay kickbacks to members) systematically billing for unnecessary or fictitious services	Graph analysis of referral patterns; tight clustering of member-provider relationships; sudden geographic concentration of billing	Graph neural networks on provider-member bipartite graph; community detection (Louvain algorithm) to identify tight clusters	South Florida home health rings; auto accident chiropractor networks billing for 60-visit courses on every PIP claim
Identity Theft / Member Fraud	Using another member's insurance credentials to receive services	Services received in geographically impossible locations; member age/sex mismatch with billed services; multiple members sharing address or phone	Anomaly detection on member service geography; biometric inconsistency flags	Member sells insurance card; services claimed in two states same day

Machine Learning for FWA: Supervised vs. Unsupervised

The fundamental tension in FWA ML is that you have limited labeled data. Confirmed fraud cases from your SIU (Special Investigations Unit) represent a tiny fraction of your claims, creating severe class imbalance. You cannot train a supervised model on 0.01% positive rate without sophisticated handling.

Supervised approaches work best when you have a meaningful labeled dataset of confirmed FWA cases. XGBoost and gradient boosting models consistently outperform neural networks on tabular claims data — the structured, sparse feature space with known semantics favors tree-based methods. Feature engineering is the critical differentiator: raw claim fields are insufficient. You need engineered features like "provider's ratio of billed-to-allowed vs. specialty peers," "member's count of providers

seen in rolling 90 days," and "time delta between service date and claim submission." SMOTE and cost-sensitive learning handle class imbalance, but neither fully solves the problem when positive rate is below 0.1%.

Unsupervised approaches — isolation forests, autoencoders, LOF (Local Outlier Factor) — do not require labeled fraud cases. They learn the distribution of normal claims and score outliers. The limitation is precision: high outlier scores are not synonymous with fraud. An unsupervised FWA model is best used as a triage tool for human investigators, not as a decision engine. Clover Health's approach to provider analytics combines unsupervised clustering to identify unusual practice patterns with human review to confirm actionability.

Graph-based methods are underused but highly effective for collusion detection. Build a bipartite graph where nodes are providers and members, and edges are claims. Dense subgraphs — providers who see the same unusual cohort of members — are collusion signals. Providers who refer exclusively within a tight cluster may indicate a kickback ring. GraphSAGE and community detection algorithms (Louvain, Label Propagation) scale to tens of millions of claim edges. This is genuinely proprietary — external FWA vendors cannot run graph analytics on your specific provider network topology.

Provider Performance Scoring and Network Optimization

Network adequacy and network optimization are different problems. Network adequacy (do you have enough providers within 30 miles for each specialty?) is a compliance requirement. Network optimization (which providers deliver the best outcomes at the lowest cost?) is a competitive differentiator. Oscar Health's narrow network strategy in its early NYC market was built on exactly this analysis — identify the 20% of providers delivering 80% of high-quality, cost-efficient care, and design a network around them.

Provider scorecards require risk-adjusted comparisons. A primary care physician serving a high-complexity population will have higher PMPM than one serving young healthy members — raw cost comparison is misleading. You must risk-adjust using HCC scores or episode grouper software (3M's ETG, Optum's Episode Treatment Groups) before comparing providers. Quality metrics — HEDIS compliance rates, readmission rates, preventive care rates — complete the picture.

Narrow network design is one of the highest-leverage decisions a health plan makes. A network that eliminates the 15% of providers with poor quality-to-cost ratios can reduce PMPM by $30-80 while maintaining or improving outcomes. However, this requires sophisticated member impact modeling — you cannot terminate a provider who serves 40% of a large employer group's members without a transition plan. Collective Health's employer-sponsored health plan model requires exactly this kind of network modeling to deliver on cost-efficiency promises to self-insured employers.

> 💡 CTO Talking Point: Provider performance scoring is where your claims data moat becomes a network contracting weapon. When you can show a hospital system that their 30-day readmission rate is 18% vs. a peer average of 11%, you have leverage in rate negotiations. When you can show that Surgeon A's episode cost for total knee replacement is $28,000 vs. Surgeon B's $41,000 with equivalent outcomes, you have the data to design tiered networks. This analysis requires 24-36 months of clean, risk-adjusted claims history. Start building it on day one.

AI and the Build vs. Buy Calculus

The most important strategic decision a health-tech CTO makes about AI is not which model to use — it's which problems are commodities and which are genuine sources of differentiation. Getting this wrong burns millions in engineering time on problems that vendors have already solved, or alternatively, outsources the capabilities that make your product unique.

Undifferentiated AI — buy: Clinical NLP for ICD-10/CPT extraction from clinical notes. Medical coding accuracy has been commoditized by Nuance, 3M, Optum360. The training data (millions of coded clinical notes) and domain expertise required to build competitive models is beyond most startups. Similarly, NLP for prior auth clinical criteria extraction, drug-drug interaction checking, and formulary management logic. These are solved problems with mature vendors. Buying them lets your ML team focus on proprietary problems.

Proprietary models on your data — build: Risk stratification models trained on your specific member population. FWA detection models trained on your specific provider network graph. Member engagement propensity models for care management outreach.

Auto-adjudication confidence scoring calibrated to your specific plan design rules. These models depend on your longitudinal claims history, your member demographics, your provider network topology. A third-party vendor cannot build them for you with the same fidelity, because they don't have your data. This is where Gravie, Justworks, and Gusto's data assets become compounding advantages — every additional year of claims history makes the models better, and competitors cannot buy their way to equivalent training data.

The operational implication: structure your data platform to enable both. Your proprietary models need clean feature stores, reproducible training pipelines, model registries, and drift monitoring. Your purchased AI tools need clean API integration, audit logging, and override mechanisms. The architecture that supports both is a unified data platform with a model serving layer — not two separate systems bolted together.

MODULE 8 **Modernization Playbook for a CTO** ▶

Most health-tech CTOs inherit a system, they don't greenfield one. Whether you're joining a benefits administration platform like Gusto or Justworks that's building claims capability, or taking over engineering at a TPA that's been running mainframe-based adjudication since 1997, the challenge is the same: modernize without breaking a system that processes payroll-critical healthcare claims for real members. This module is an opinionated, sequenced playbook for that challenge.

Assessing Current State: The Technical Debt Inventory

Before you write a single line of new code, you need a brutally honest inventory of what you have. This is not an exercise in producing a PowerPoint for the board — it's a prerequisite for making defensible architectural decisions. Skipping this step is why most modernization programs fail: they address symptoms (slow UI, manual workarounds) rather than root causes (broken data model, absence of event sourcing, hard-coded plan design logic).

The technical debt inventory has four components. First, system topology: what are all the systems involved in the claims lifecycle? Map data flows between them. Where are the synchronous dependencies that create fragility? Which integrations are

undocumented or maintained by one person? Second, auto-adjudication rate audit: pull claims data for the last 12 months and calculate auto-adj rate by claim type (professional, institutional, pharmacy), by payer, and by denial reason. An auto-adj rate below 60% on professional claims is a serious problem. A rate below 80% on pharmacy claims is a critical failure. Document where the leakage is. Third, stakeholder interviews: talk to claims processors, member services reps, provider relations, and compliance. They know where the system fails — they've built workarounds around every broken edge case. These workarounds are your technical debt map. Fourth, dependency and vendor audit: identify every vendor contract, SLA, and API dependency. A legacy TPA running on a TriZetto Facets or QNXT installation has a very different modernization path than one running on homegrown Java — not because one is better, but because the migration risk profile is different.

The output of this assessment is a prioritized technical debt register with three columns: cost of inaction (what does this debt cost in manual labor, error rate, or compliance risk per year?), cost of remediation (what does fixing it cost?), and risk of breakage during remediation (how likely is a fix to cause downstream failures?). This register drives your roadmap sequencing and your budget asks.

Phase 1: Quick Wins (0–6 Months)

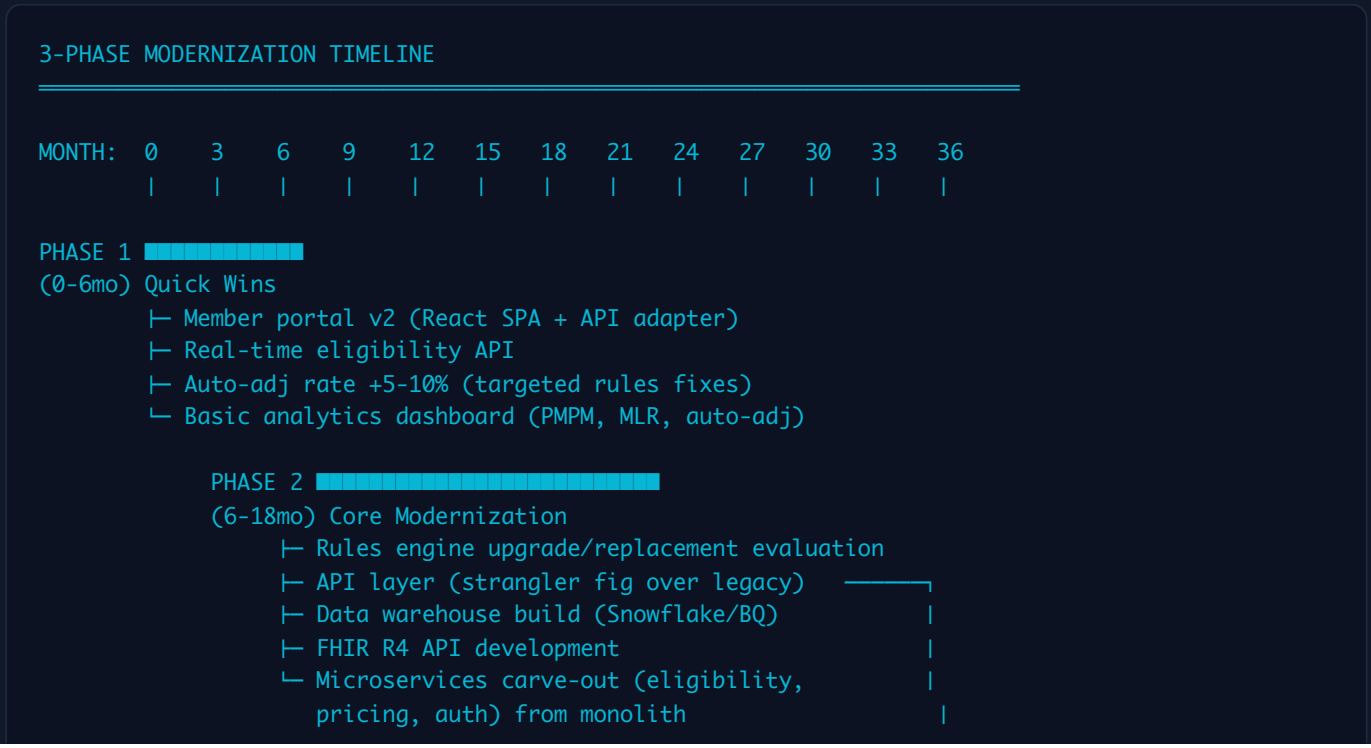
Phase 1 is about demonstrating momentum, building team confidence, and establishing the data and API foundations that Phase 2 requires. You should not attempt a rules engine replacement or a data warehouse build in Phase 1. The goal is tangible, visible improvement delivered quickly.

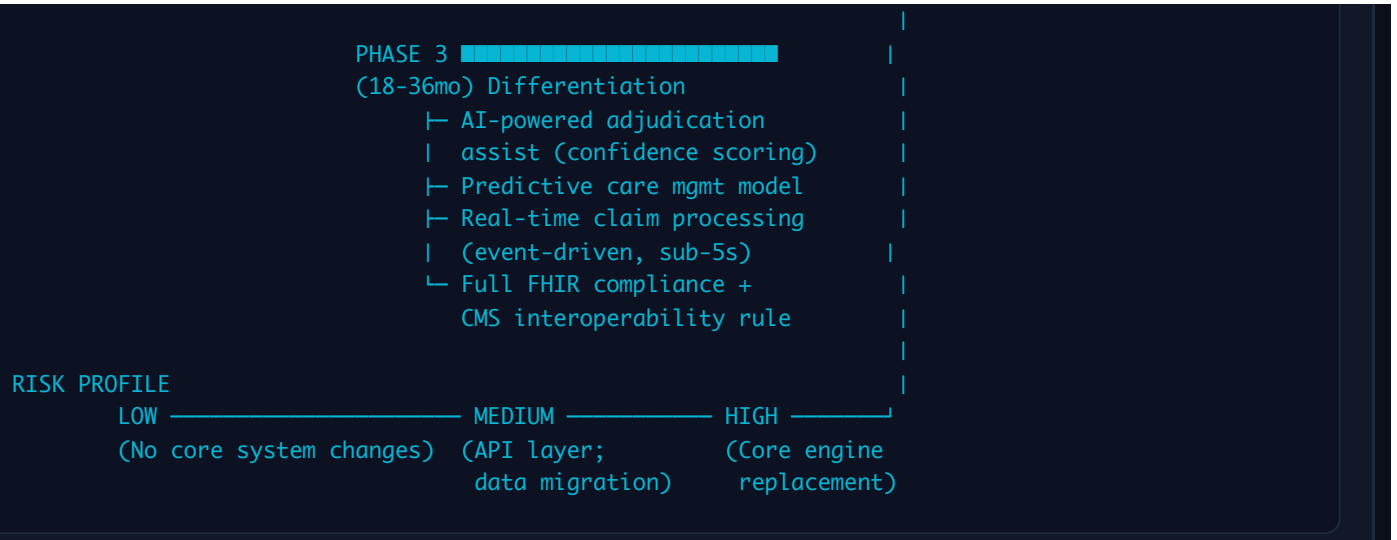
Member portal redesign is often the most visible win. Legacy member portals are typically server-rendered, session-based applications with poor mobile performance and no real-time data. A React-based SPA backed by your existing claims data via a new API layer can dramatically improve NPS within 90 days. Critically, do not use the portal redesign as an opportunity to rebuild the backend — build a thin API adapter over existing systems and ship the new frontend first. Oscar Health's member experience advantage was built on exactly this discipline: ship a best-in-class mobile experience while legacy backend migration happens behind the scenes.

Real-time eligibility API is a high-leverage technical investment that makes every downstream system better. Provider portals, member-facing apps, customer service tools, and claims adjudication all benefit from reliable, low-latency eligibility verification. Migrating eligibility from batch file exchange (834 transactions processed nightly) to a real-time REST API with event-driven updates transforms the operational experience for providers and members. This also forces you to build the event sourcing infrastructure for member data that Phase 2's data warehouse will depend on.

Auto-adjudication rate improvement of 5-10 percentage points is achievable in Phase 1 through targeted rules work rather than engine replacement. Audit your top 20 denial reason codes by volume. For each, determine: is this denial correct (genuinely not payable)? Or is it a rules gap, a data quality issue, or a missing configuration? Typically, 30-40% of high-volume denial codes are configuration or data quality problems that can be fixed in the existing rules engine. A 5-point improvement in auto-adj rate on a plan processing 50,000 claims/month is 2,500 claims no longer requiring manual review — roughly 2-3 FTEs of claims processor time.

Basic analytics dashboard — surfacing PMPM, MLR, auto-adj rate, and utilization metrics in a tool that operations, finance, and clinical leadership can access without a BI analyst — creates organizational alignment around data. Use a simple data stack for Phase 1: dbt on your existing data warehouse (or a simple Redshift/BigQuery instance), Metabase or Looker for visualization. Do not build custom analytics tooling in Phase 1. Ship something useful fast.





Phase 2: Core Modernization (6–18 Months)

Phase 2 is where you make irreversible architectural decisions. The two most consequential are: (1) do you replace your rules engine or upgrade it? and (2) how aggressively do you apply the strangler fig pattern to your legacy monolith?

Rules engine evaluation: if your current engine is a major vendor (TriZetto, NICE Systems, Edifecs), the upgrade path is well-defined but expensive and slow. Replacement with a modern engine (Drools, or a custom-built configuration-driven system) is faster long-term but carries significant migration risk — you must port thousands of rules without regression. The decision criterion is not technical, it's operational: can your team run a parallel adjudication environment for 6 months to validate equivalence? If yes, replacement is viable. If not, upgrade first and plan replacement for Phase 3.

The **strangler fig pattern** is the safest approach to legacy migration in healthcare IT. Rather than rewriting the monolith, you build a new API layer in front of it and route new features through the new layer. Over time, you reimplement legacy capabilities as new microservices and route traffic away from the old system. The key discipline: every new capability must be built in the new architecture. Never add features to the monolith during Phase 2. Collective Health built its TPA platform entirely in the new architecture behind an adapter layer over partner carrier systems — an extreme version of this pattern, but demonstrably viable.

Data warehouse build: Phase 2 is when you stop tolerating ad-hoc SQL queries against production transactional databases and build a proper analytical data layer. The modern data stack — Airbyte/Fivetran for ingestion, Snowflake or BigQuery as the warehouse, dbt

for transformation, Hightouch for reverse ETL — is well-understood and cost-effective. The critical healthcare-specific requirement is HIPAA-compliant data access controls: column-level security on PHI fields, audit logging on all queries touching member data, and a de-identification pipeline for analytics use cases that don't require patient-level identification.

FHIR API development: the CMS interoperability rule mandates FHIR R4 Patient Access and Provider Directory APIs for Medicare Advantage and Medicaid plans. For commercial plans, FHIR is increasingly a competitive requirement — employers buying self-insured plans want to feed claims data into their population health platforms. Building FHIR APIs in Phase 2 positions you for Phase 3's real-time processing architecture and avoids a scramble when CMS extends interoperability mandates to commercial plans (widely anticipated).

Phase 3: Differentiation (18–36 Months)

Phase 3 is where the technology investments compound into competitive advantage. By this point, you have clean data, a modern API architecture, and a team that has shipped two phases successfully. Now you build the capabilities that define your product roadmap for the next five years.

AI-powered adjudication assist is not full AI adjudication — it's a confidence scoring layer on top of your rules engine. The model outputs a confidence score for each adjudication decision. High-confidence decisions (>95%) auto-adjudicate without review. Medium-confidence decisions (70-95%) get routed to experienced examiners with the model's reasoning surfaced as decision support. Low-confidence decisions get full manual review. This architecture improves auto-adj rate without sacrificing accuracy — the model handles easy cases, humans handle ambiguous ones. Gusto's payroll-adjacent claims processing for SMB employers benefits enormously from this because their claim volume is high but their examiner team is lean.

Real-time claim processing requires event-driven architecture end-to-end: claims submitted via real-time 837 transaction or REST API, eligibility verified synchronously, clinical edits applied via rules engine in-process, and an adjudication decision returned within 5 seconds for clean claims. This is a massive architectural change from the batch-

processing paradigm of legacy systems, but it's the future of the market — providers want to know at point of service whether a claim will pay.

Engineering Team Structure

A health-tech CTO building a claims platform needs a deliberately structured team. The skills required are not interchangeable, and standard software engineering hiring pipelines will not surface the domain expertise you need.

Claims engineers: 3-5 engineers with deep EDI (X12 837/835/834/270/271) expertise and healthcare business domain knowledge. These are not backend engineers who can learn healthcare — they are healthcare technologists who know how to build software. They are rare and expensive. Invest in retention aggressively.

Compliance and security engineers: HIPAA, state insurance regulations, CMS program requirements, and SOC 2 compliance generate continuous engineering work. You need at least 2 engineers who treat regulatory compliance as a first-class engineering concern, not a checklist item. These engineers should be embedded in the architecture review process, not consulted after decisions are made.

Data engineers: 4-6 engineers focused on the data platform — ingestion, transformation, warehousing, and data quality. In a claims system, data quality is a patient safety issue: incorrect eligibility data causes claim denials that delay care. Your data engineers must understand the healthcare data model (member, claim, provider, plan benefit), not just generic data pipeline patterns.

ML engineers: 3-4 engineers building and operating the predictive models described in Module 7. This team needs a mix of ML expertise (model development, feature engineering, experiment tracking) and MLOps capability (serving infrastructure, drift monitoring, retraining pipelines). At Gravie's scale, this team's work on risk stratification and care management directly influences clinical outcomes for members.

Member experience frontend engineers: 3-5 engineers building member-facing products. Healthcare UX is unusually hard because you're explaining complex financial and clinical concepts (EOBs, deductibles, prior authorizations) to non-expert users under stress. These engineers should be paired with UX research capability.

Integration specialists: 2-3 engineers managing the partner ecosystem — carrier integrations, clearinghouse connections, pharmacy benefit manager APIs, EHR integrations. These are the engineers who speak EDI and HL7 fluently and maintain relationships with vendor technical teams.

QA with healthcare domain knowledge: 2-3 QA engineers who understand the clinical and operational consequences of bugs. A QA engineer who doesn't understand that an incorrect COB (Coordination of Benefits) calculation will cause incorrect member cost-sharing for every claim where a member has dual coverage cannot write meaningful test cases for that feature.

> 💡 CTO Talking Point: The engineering org structure above is 20-25 people for a mid-market claims platform. That's a $4-6M annual engineering investment. The board will ask: why can't we buy this from TriZetto or a white-label TPA and avoid the build cost? The answer is that you can — and you should for the commodity capabilities. The engineering investment is for the 20% of capabilities that are genuinely differentiating: your member experience, your analytics, your AI models, your employer integration layer. Be specific about which engineering investments are moat-building and which are table stakes.

Vendor Evaluation Framework

Health-tech vendor evaluation is systematically worse than it should be. Procurement processes favor vendors who are good at selling over vendors who deliver. The following weighted scoring framework is designed to correct for this by forcing quantitative assessment of operational and technical criteria that vendors routinely obscure.

Evaluation Criterion	Weight	What to Test / Verify	Red Flags	Score 1-5	Weighted Score
Auto-adjudication rate	15%	Request 12-month auto-adj rate by claim type from 3 reference customers similar to your volume/mix. Refuse aggregate rates — get professional and institutional separately.	Vendor cannot provide reference customer data; rate below 75% on professional claims	—	—

Evaluation Criterion	Weight	What to Test / Verify	Red Flags	Score 1-5	Weighted Score
API maturity and integration flexibility	15%	Require a real API sandbox. Test actual EDI ingestion, real-time eligibility, and claim status endpoints. Count undocumented limitations.	No sandbox; REST "API" is actually SFTP file drop with JSON wrapper; rate limits incompatible with your volume	—	—
HIPAA and security posture	15%	BAA terms (particularly breach notification SLA and indemnification), SOC 2 Type II report (not just attestation), penetration test recency, encryption at rest/in transit standards	SOC 2 report older than 18 months; no BAA indemnification; encryption exemptions for legacy data stores	—	—
Plan design configurability	12%	Can your team configure plan designs without vendor professional services? Test HSA-compatible HDHP, HRA-integrated plan, and tiered network benefit simultaneously.	Every plan design change requires a vendor ticket; configuration UI is COBOL screen forms; no self-service benefit administration	—	—
Data access and portability	12%	Full data export on contract termination (within 30 days, in standard format), real-time data feed for your warehouse, no data licensing fees for your own data	Data export limited to PDF reports; termination data export takes 6+ months; proprietary data format with no standard export	—	—
Scalability and SLA	10%	99.9% vs 99.95% uptime SLA (the	No financial penalties in	—	—

Evaluation Criterion	Weight	What to Test / Verify	Red Flags	Score 1-5	Weighted Score
		difference is 4x more downtime annually); claims processing SLA during open enrollment peaks; disaster recovery RTO/RPO	SLA; "commercially reasonable efforts" language in uptime commitments; RTO > 24 hours		
Regulatory compliance roadmap	10%	When did they ship FHIR R4 Patient Access API? What is their timeline for CMS prior auth rule (January 2026)? Do they have a dedicated compliance engineering team?	FHIR compliance is "on roadmap" without timeline; compliance features ship after regulatory deadline, not before	—	—
Implementation track record	8%	Require 5 customer references at your scale. Ask specifically: what was the go-live date vs. contracted date? What went wrong?	All references are enterprise customers when you're mid-market; no reference will give a candid implementation assessment	—	—
Total cost of ownership	8%	PMPM pricing including professional services for plan design changes, integration work, and regulatory updates. Many vendors quote low PMPM and charge $300/hr for everything else.	Pricing structure requires professional services for standard configuration; "unlimited" plan design has 20-page definition of what's excluded	—	—
Vendor financial stability	5%	Last fundraise date and amount, ARR	Undisclosed funding; single	—	—

Evaluation Criterion	Weight	What to Test / Verify	Red Flags	Score 1-5	Weighted Score
		growth trajectory, customer concentration (is one customer >25% of revenue?), key person dependency	large customer concentration; CTO/founder departure in last 12 months		

Key Metrics for a Claims System CTO

You cannot manage what you do not measure, and the wrong metrics create perverse incentives. The following dashboard is what a claims system CTO should review weekly — not because these are the only metrics, but because each one is a leading indicator of a distinct failure mode.

Metric	Definition	Target / Benchmark	Failure Mode It Detects	Review Frequency
Auto-adjudication rate	% of claims adjudicated without human intervention	>85% professional; >95% pharmacy; >70% institutional	Rules engine configuration gaps, data quality degradation, new claim patterns not handled	Daily
Claims turnaround time	Median days from receipt to final payment/denial, by claim type	<14 days clean claims; <30 days pended; state prompt-pay compliance	Processing bottlenecks, manual queue backup, system performance issues, compliance risk	Daily
PMPM admin cost	Total claims operations cost divided by member-months	$15-25 PMPM for mid-market TPA; <$10 PMPM at scale (>500K members)	Automation leverage, FTE efficiency, technology investment return	Monthly
Member NPS	Net Promoter Score from member experience surveys, segmented by claims interaction	>30 for health insurance (industry average is near 0); Oscar targets >50	Member experience failures in EOB clarity, portal usability, dispute resolution	Monthly

Metric	Definition	Target / Benchmark	Failure Mode It Detects	Review Frequency
Provider payment accuracy	% of provider payments made at contractually correct amount; overpayment and underpayment rates tracked separately	>99% payment accuracy; <0.5% overpayment rate	Repricing logic errors, fee schedule mismatches, COB calculation errors	Weekly
System uptime (claims processing)	% availability of core claims processing, eligibility, and member portal systems	>99.9% for claims processing; >99.5% for member portal; measure separately from batch windows	Infrastructure reliability, deployment risk, third-party dependency failures	Real-time / Weekly
First-call resolution rate	% of member and provider service calls resolved without callback or escalation	>75% first-call resolution; <3% escalation to supervisor	Tooling gaps for service agents, data access latency, claims transparency failures	Weekly
FWA recovery rate	Dollars recovered or avoided through FWA detection as % of total medical spend	0.5-2% of medical spend; benchmark against NHCAA industry data	Detection model performance degradation, SIU capacity, recovery workflow gaps	Monthly
FHIR API SLA compliance	% of FHIR API requests served within regulatory SLA (CMS requires <500ms response time for Patient Access API)	>99% of requests within 500ms; <0.1% error rate	Regulatory compliance risk, infrastructure scaling gaps, data model query performance	Daily

Metric	Definition	Target / Benchmark	Failure Mode It Detects	Review Frequency
ML model performance (FWA + risk)	Precision/recall for FWA detection model; Population Stability Index for risk stratification model	FWA precision >40% at 80% recall; PSI <0.2 (significant drift threshold)	Model drift, feature pipeline failures, data distribution shifts from network changes	Weekly

Board-Level Narrative: The 3-Year Technology Roadmap

Boards of health-tech companies are populated by financial investors and experienced operators who care about four things: cost reduction, risk mitigation, competitive advantage, and regulatory compliance. They do not care about microservices, Kafka, or FHIR as concepts. They care about what those investments produce in business terms. If you present a technology roadmap without translating every initiative into one of these four categories, you will lose the room.

Cost reduction: Every auto-adjudication rate improvement translates directly to FTE cost avoidance. A 10-point improvement in auto-adj rate on 100,000 monthly claims is 10,000 fewer manual reviews, equivalent to 8-10 FTEs at $65K fully-loaded cost — $520,000-$650,000 in annual savings. Present this math explicitly. The PMPM admin cost reduction from $22 to $15 over three years on 200,000 members is $16.8M in cumulative savings. These numbers make technology investment credible to a finance-oriented board.

Risk mitigation: Legacy system risk is existential for a claims processor. A system outage during open enrollment, a HIPAA breach, or a CMS audit finding can generate fines, member churn, and regulatory sanctions that dwarf the modernization cost. Present the risk inventory explicitly: "Our current system has X single points of failure, Y undocumented dependencies, and Z compliance gaps against the CMS prior auth rule effective January 2026." Make the cost of inaction concrete.

Competitive advantage: Gravie's ICHRA-first model, Oscar's vertical integration, and Clover Health's data-driven provider partnerships are all technology stories. Your board

needs to understand why your technology roadmap creates durable competitive advantage — not just operational efficiency. "Our AI-powered member experience will achieve NPS 20 points above industry average" is a board narrative. "We're rewriting the member portal in React" is not.

Regulatory compliance: The CMS prior authorization rule (effective January 2026 for most payer types), the CMS interoperability rule, state surprise billing implementations, and No Surprises Act compliance all generate mandatory technology investments. Present these as non-negotiable commitments with clear timelines and cost estimates. Board members with insurance industry experience will respect a CTO who understands the regulatory calendar — it signals that you understand the operating environment, not just the technology.

> 💡 CTO Talking Point: The board presentation that lands is one where every Phase 1, 2, and 3 initiative maps to a specific business outcome metric that the board already monitors: MLR improvement, admin cost PMPM, member NPS, or regulatory fine avoidance. Build a slide that is literally a 3x4 matrix: phases on one axis, board-level concerns (cost, risk, competitive, regulatory) on the other, with the specific quantified impact in each cell. This reframes technology investment from "spending" to "portfolio allocation" — a concept the board understands.

Your First 90 Days: CTO Checklist

This checklist is ordered by urgency, not importance. Some items are urgent because delays compound — a HIPAA compliance gap identified on day 5 is addressable; one identified on day 60 has probably already been exploited. Others are important because they determine the quality of every decision you make for the next three years.

Days 1-30: Listen and assess

- Sit with claims examiners for 4 hours minimum. Watch them work. Document every manual workaround.
- Pull auto-adjudication rate by claim type and denial reason code from the last 12 months. This single analysis will tell you more about system health than any architecture diagram.

- Read every vendor contract. Note data portability terms, termination clauses, and SLA financial penalties. Flag any contract that lacks meaningful remedies.
- Audit HIPAA BAA coverage: every vendor who touches PHI must have a current BAA. Gaps must be closed within 30 days.
- Review the last SOC 2 report or HIPAA risk assessment. Identify open findings and their remediation status.
- Interview the top 5 performing engineers and the bottom 5 by tenure. The institutional knowledge concentration you discover will determine your bus factor risk.
- Map every regulatory deadline in the next 24 months: CMS prior auth rule, state surprise billing amendments, FHIR interoperability requirements. Each one needs an owner and a budget estimate.
- Assess current FHIR compliance status: Patient Access API, Provider Directory API, Drug Formulary API. If any are missing and you have Medicare Advantage or Medicaid business, this is a compliance fire.

Days 31-60: Diagnose and plan

- Produce the technical debt register (cost of inaction, cost of remediation, risk of breakage). Get alignment from operations and finance leadership on prioritization.
- Establish baseline metrics for the CTO dashboard above. You cannot improve what you cannot measure, and you need 60 days of baseline before you can claim improvement.
- Evaluate the top 3 denial reason codes for addressability. Identify quick wins for auto-adj rate improvement that can ship in Phase 1.
- Assess the data platform: can you run cohort analysis on claims history? If a business leader asks "what is our PMPM for members with diabetes in employer group X?" how long does it take to answer? More than 24 hours is a data platform problem.
- Interview 5 provider offices about their claim submission and payment experience. Provider NPS is the canary for claims system reliability — providers flag problems with the system before internal teams do.
- Evaluate team structure against the org design above. Identify critical skill gaps. Begin recruiting for the highest-leverage gaps immediately — healthcare technology recruiting takes 3-6 months.

Days 61-90: Commit and communicate

- Draft the Phase 1 roadmap with committed deliverables, owners, and success metrics. Get buy-in from the CEO and COO before presenting to the board.
- Present the technical debt register and 3-phase roadmap to the board using the cost/risk/competitive/regulatory framework above. Frame every initiative in business terms, not technology terms.
- Establish engineering team rituals: weekly metrics review (auto-adj, turnaround time, uptime), monthly architecture review, quarterly compliance review. Rituals create accountability.
- Define the build vs. buy decision framework for your specific context. Document which capabilities are moat-building (must build) and which are commodity (must buy). Every future technology decision should reference this framework.
- Ship one visible thing. Anything. A new member portal page, a real-time eligibility endpoint, a PMPM dashboard. The engineering team needs to see that the new leadership delivers — and so does the rest of the organization.